

多旋翼飞行器设计与控制
系列实验

RflySim高级版 第2讲 飞行控制算法开发

戴训华 daixunhua@163.com

中南大学 计算机学院 副教授

北京航空航天大学 自动化学院 博士



北航可靠飞行控制研究组

BUAA Reliable Flight Control Group

大纲

1. 课程学习
2. 原理详解
3. 进阶例子
4. 小结

本讲的例程源码路径：
RflySimAPIs\FlightControlExpCourse

1.课程学习

1.1 参考教材

- 全权, 戴训华, 王帅著. 多旋翼飞行器设计与控制实践[M]. 北京:电子工业出版社. 2020



注：

- 左侧的《多旋翼飞行器设计与控制实践》为2020年推出的针对飞行控制算法开发的实践课程，里面包含了部分理论知识与一系列实验，使得读者可以很快地将自己算法在Simulink中编程，并下载到Pixhawk真机中做飞行实验。
- 右侧的《多旋翼飞行器设计与控制》为2017年推出的教程，主要针对多旋翼控制理论。

1.课程学习

1.2 课程网站：https://rflysim.com

- 包含书本前四章关于平台使用相关内容
- 提供课件源码下载地址
- 提供利用本平台开发的一些项目demo
- 提供高级功能的基本介绍和使用方法
- 提供常见问题的解答

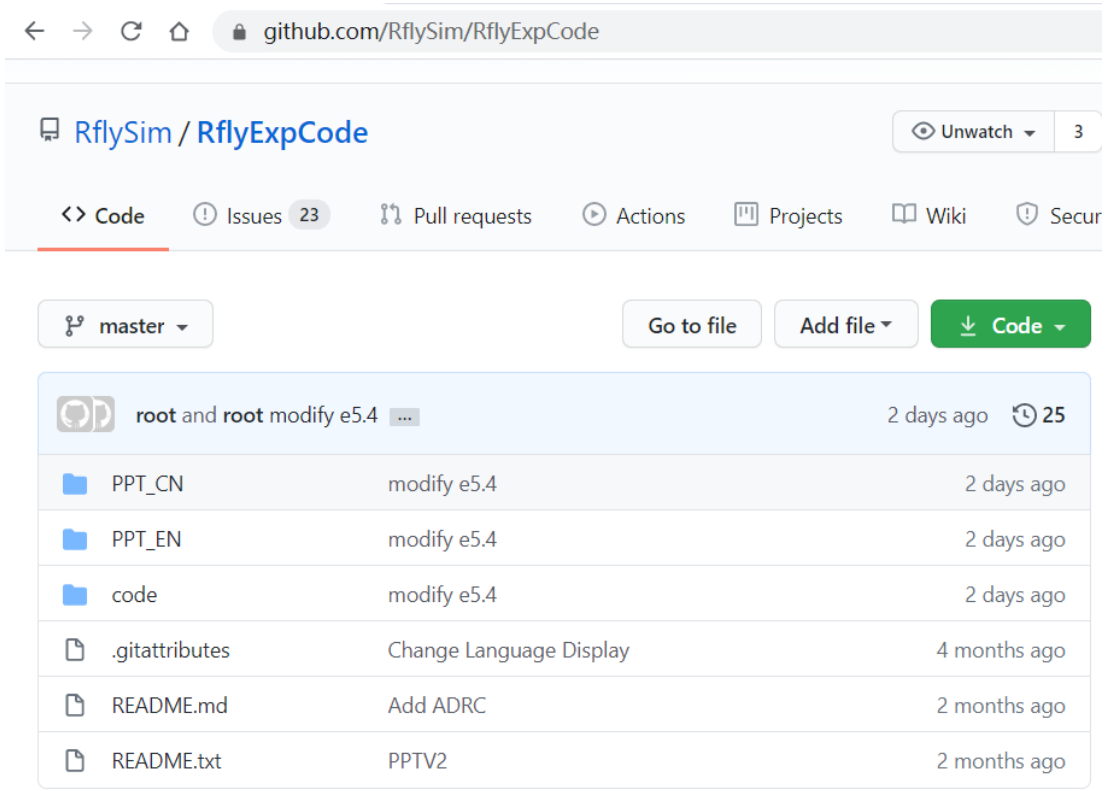


1.课程学习

1.3 最新课件/源码下载地址

- https://rflysim.com/zh/5_Course/Content.html
- https://github.com/RflySim/RflyExpCode

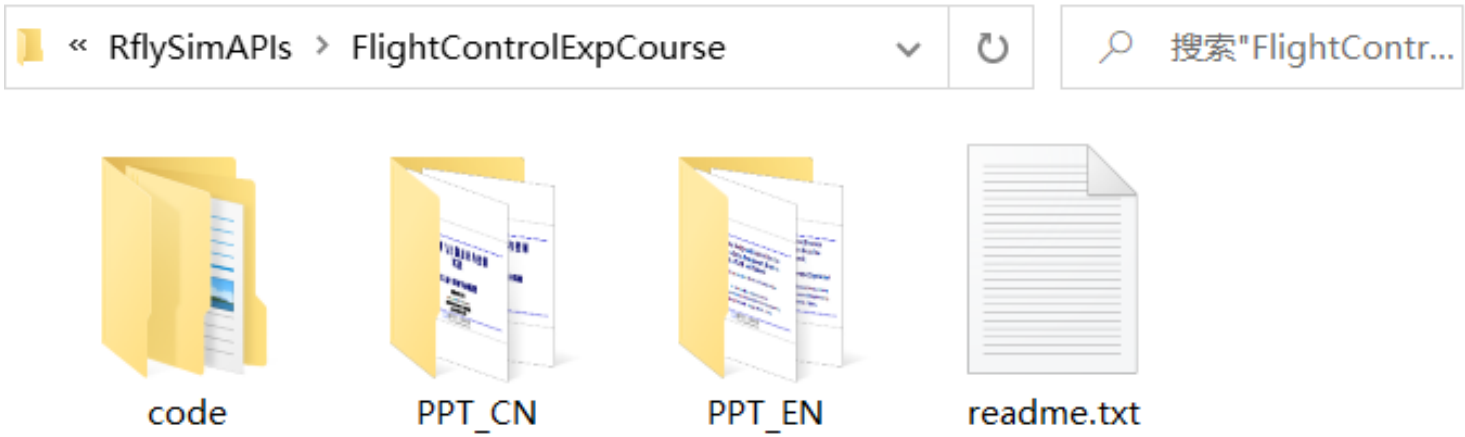


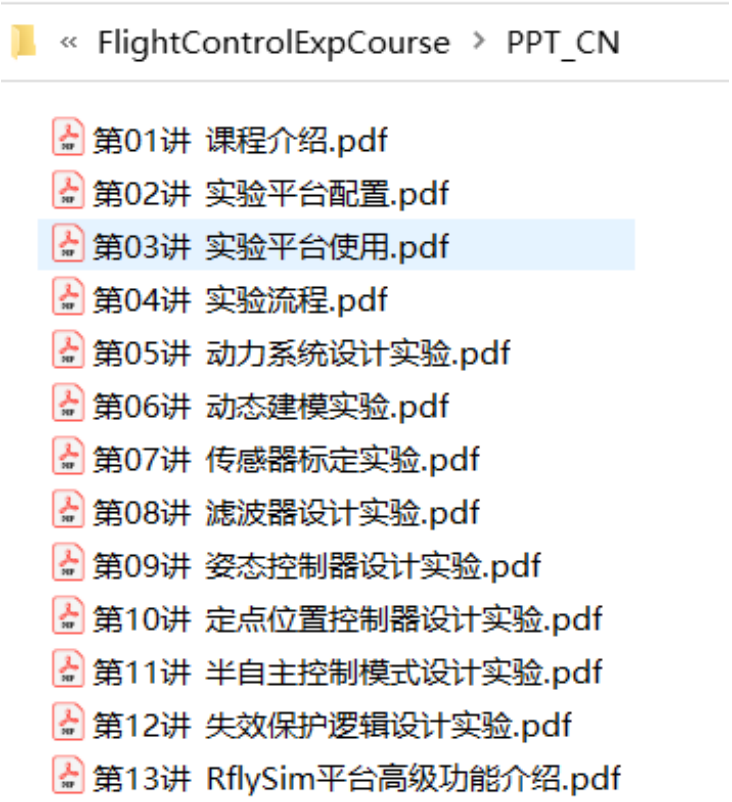


1.课程学习

1.4 课件/源码地址2

- 直接访问“RflySimAPIs\FlightControlExpCourse”文件夹
- 其中“PPT_CN”包含了课程的全部中文课件
- “code”文件夹包含了课程对应源码
- 请结合教程/PPT/源码学习飞控算法开发相关课程

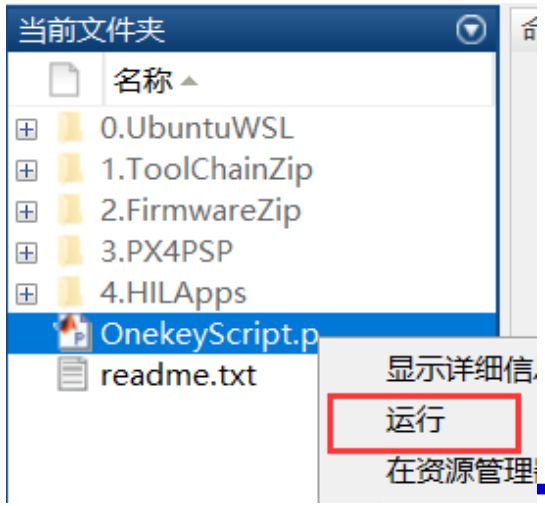


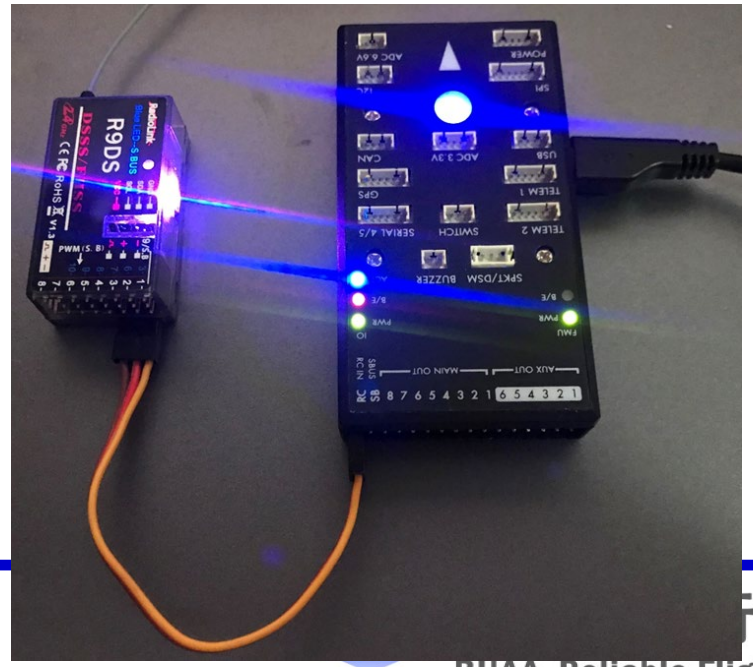


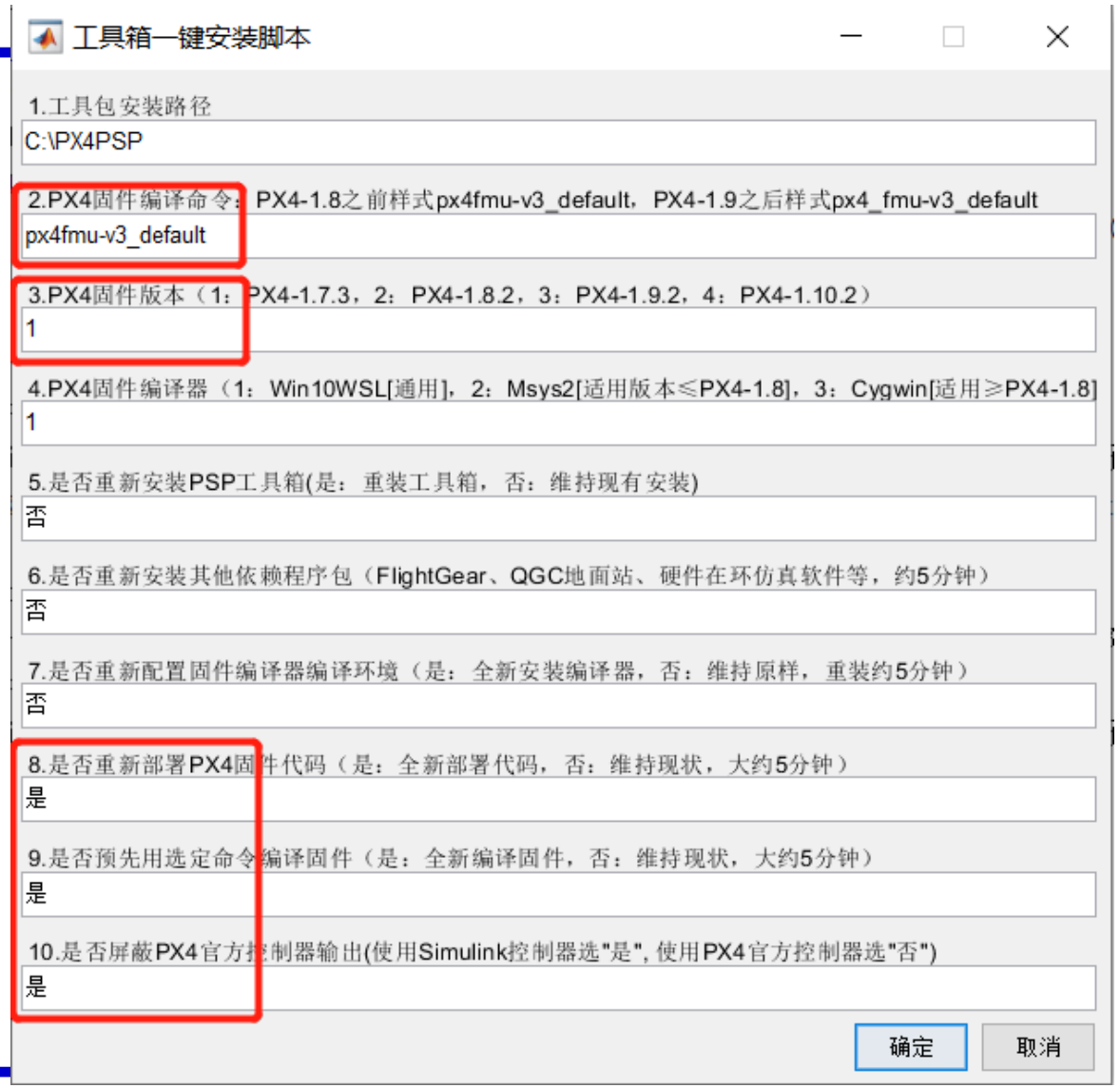
1.课程学习

1.5 重新运行安装包内“OnekeyScript.p”脚本

- 如果使用教材推荐的Pixhawk 1（外观见下图）的编译命令px4fmu-v3_default
- 使用书本推荐的PX4固件版本“1”— PX4-1.7.3
- 其他配置如右图所示。







1.课程学习

2.PX4固件编译命令：PX4-1.8之前样式px4fmu-v3_default，PX4-1.9之后样式px4_fmu-v3_default

px4fmu-v3_default

不同Pixhawk硬件编译命令不同，常见编译命令（PX4 1.9及之后固件）如下

- Pixhawk 1: px4_fmu-v2_default
- Pixhawk 1 (2M flash): px4_fmu-v3_default
- Pixhawk 4: make px4_fmu-v5_default
- Pixracer: make px4_fmu-v4_default
- Pixhawk 3 Pro: make px4_fmu-v4pro_default
- Pixhawk Mini: make px4_fmu-v3_default
- Pixhawk 2: make px4_fmu-v3_default
- mRo Pixhawk: make px4_fmu-v3_
- HKPilot32: make px4_fmu-v2_default
- Pixfalcon: make px4_fmu-v2_default
- Dropix: make px4_fmu-v2_default
- MindPX/MindRacer: make airmind_mindpx-v2_default
- mRo X-2.1: make auav_x21_default
- Crazyflie 2.0: make bitcraze_crazyflie_default
- Intel® Aero: make intel_aerofc-v1_default



Pixhawk 1 (FMUv2)
2M flash Version (FMUv3)



mRo Pixhawk (FMUv3)



Cube (Pixhawk 2, FMUv3)



Pixhawk 3 Pro (FMUv4)



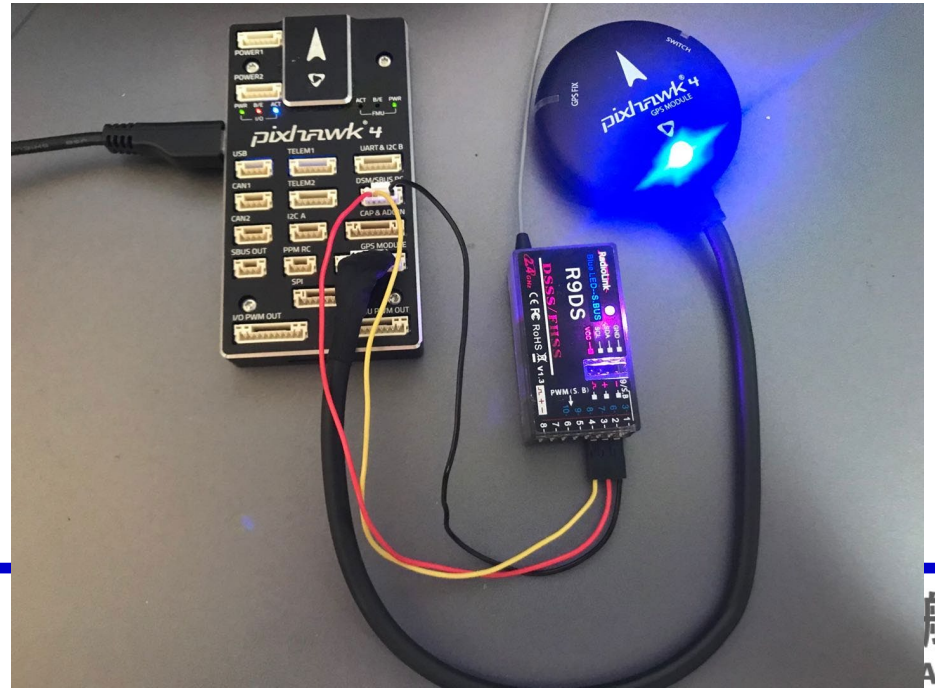
Pixhawk 4 (FMUv5)

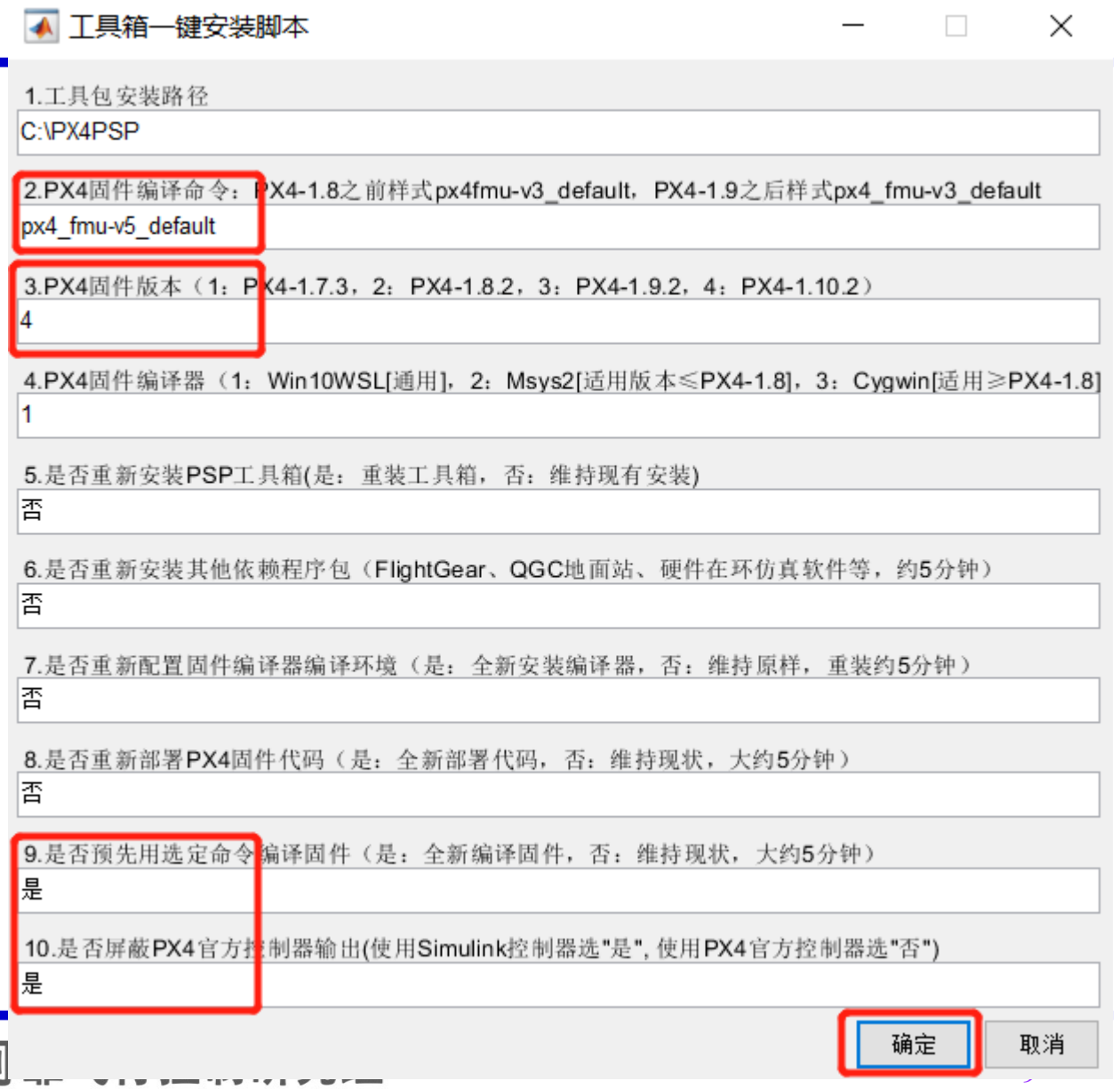


Pixhawk 4 Mini (FMUv5)

1.课程学习

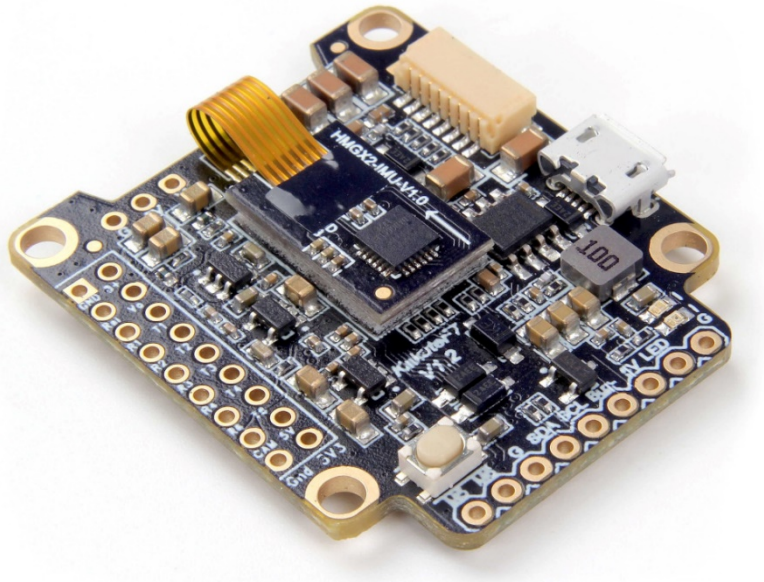
- 如果使用最新的Pixhawk 4（外观见下图）自驾仪，其编译命为令px4_fmu-v5_default
- 推荐使用最新的PX4固件版本“4”— PX4-1.10.2
- 其他配置如右图所示。

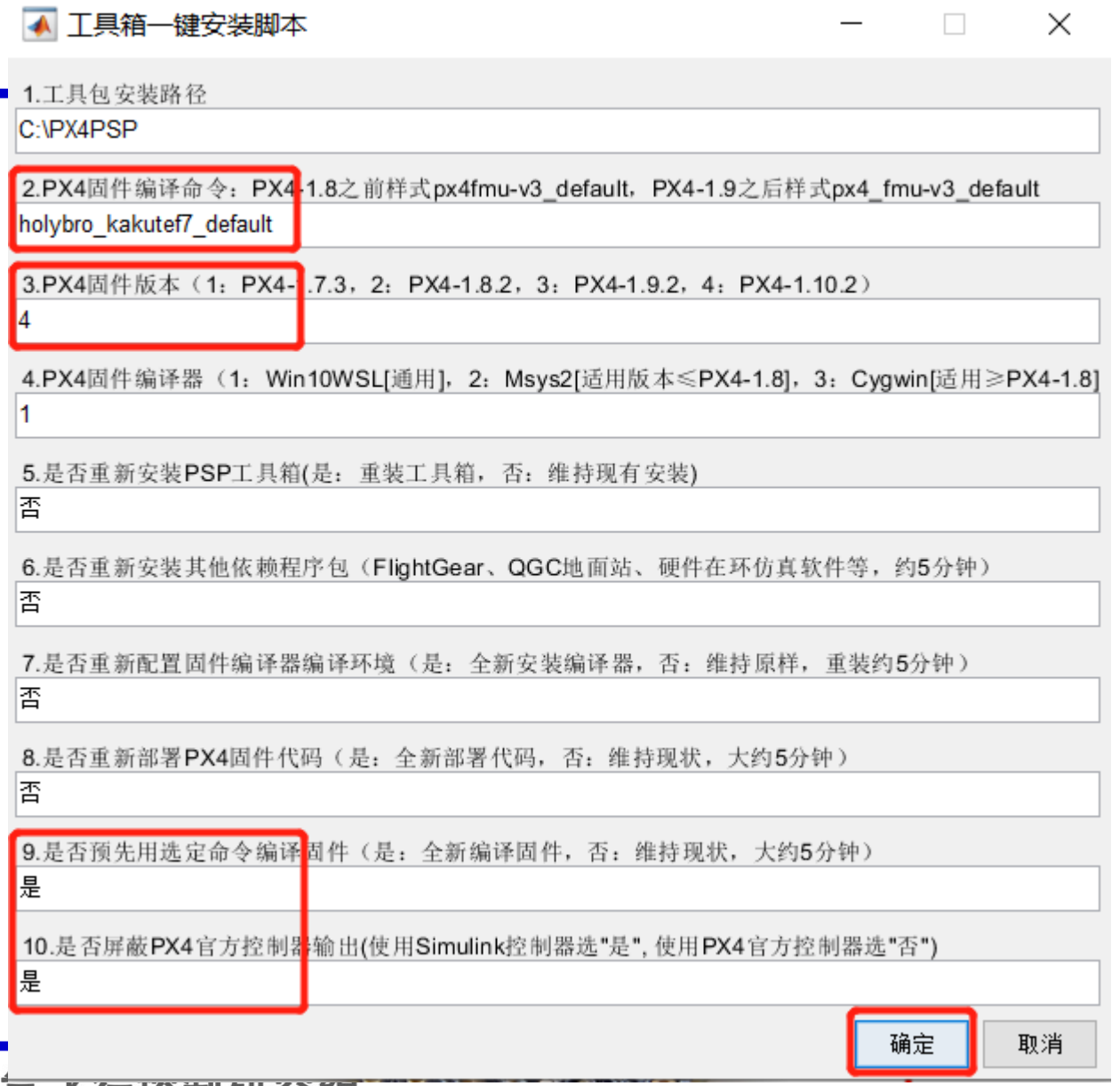




1.课程学习

- 如果使用最新的Kakute F7（外观见下图）自驾仪，其编译命为令holybro_kakutef7_default
- 推荐使用最新的PX4固件版本“4”— PX4-1.10.2.
- 其他配置如右图所示。

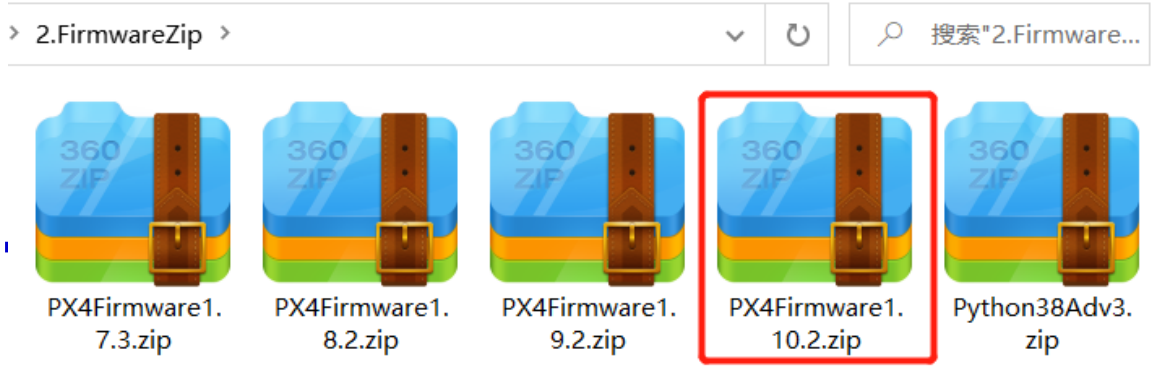


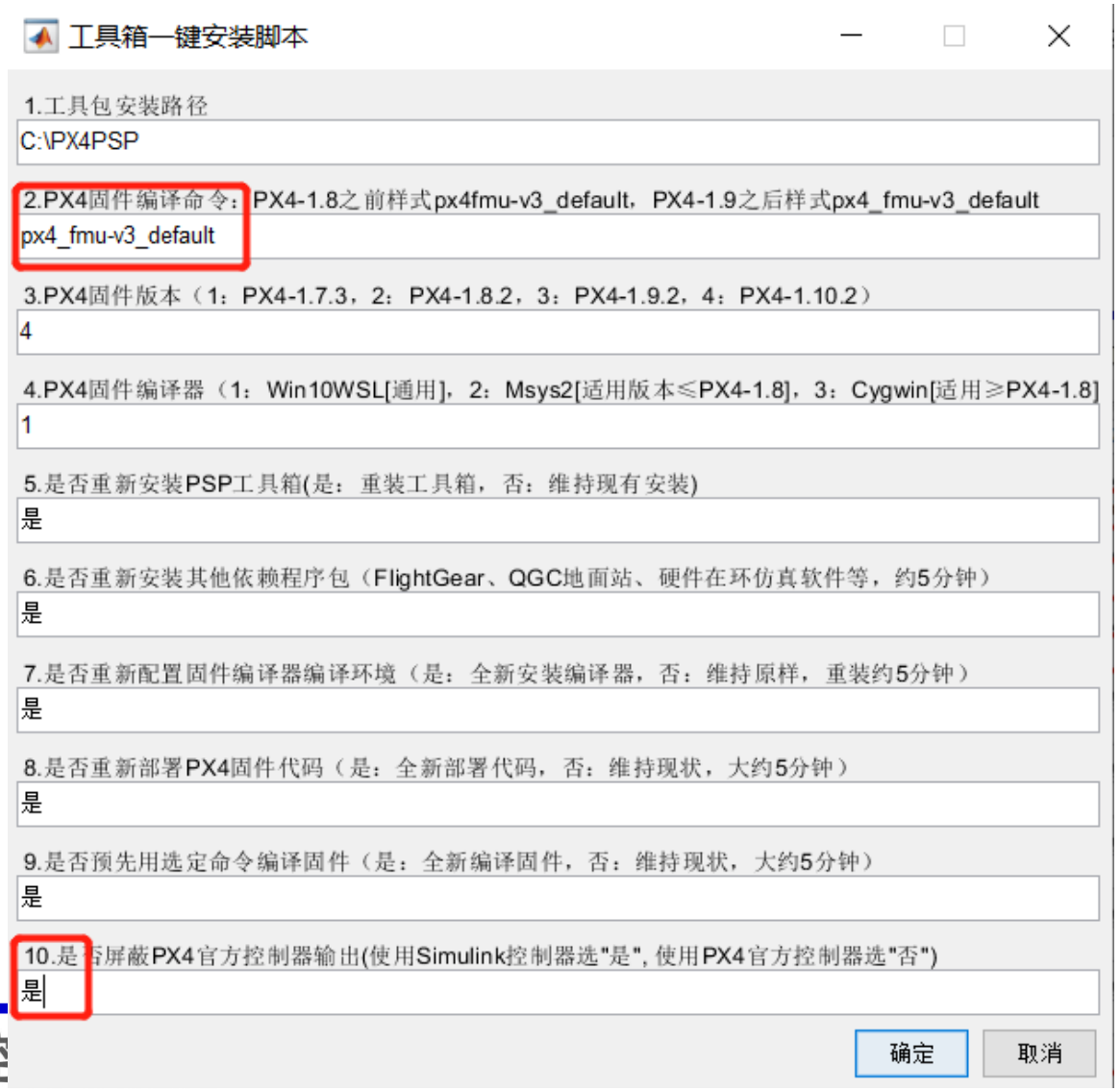


1.课程学习

1.6 使用自己的PX4源码

- 如果您需要使用自己的PX4固件代码，请将您的Firmware文件夹压缩为Firmware.zip文件，根据版本重命名（例如，如果基于PX4 1.10开发则命名为“PX4Firmware1.10.2.zip”）并替换掉“2.FirmwareZip”文件夹下的固件，在右图安装选项中的固件版本处选择“4”即可。
- 推荐使用Win10WSL编译器，因此编译器选“1”。
- 是否屏蔽PX4输出项目选择“是”，脚本会自动完成所有需要的固件修改以适配本平台





1.课程学习

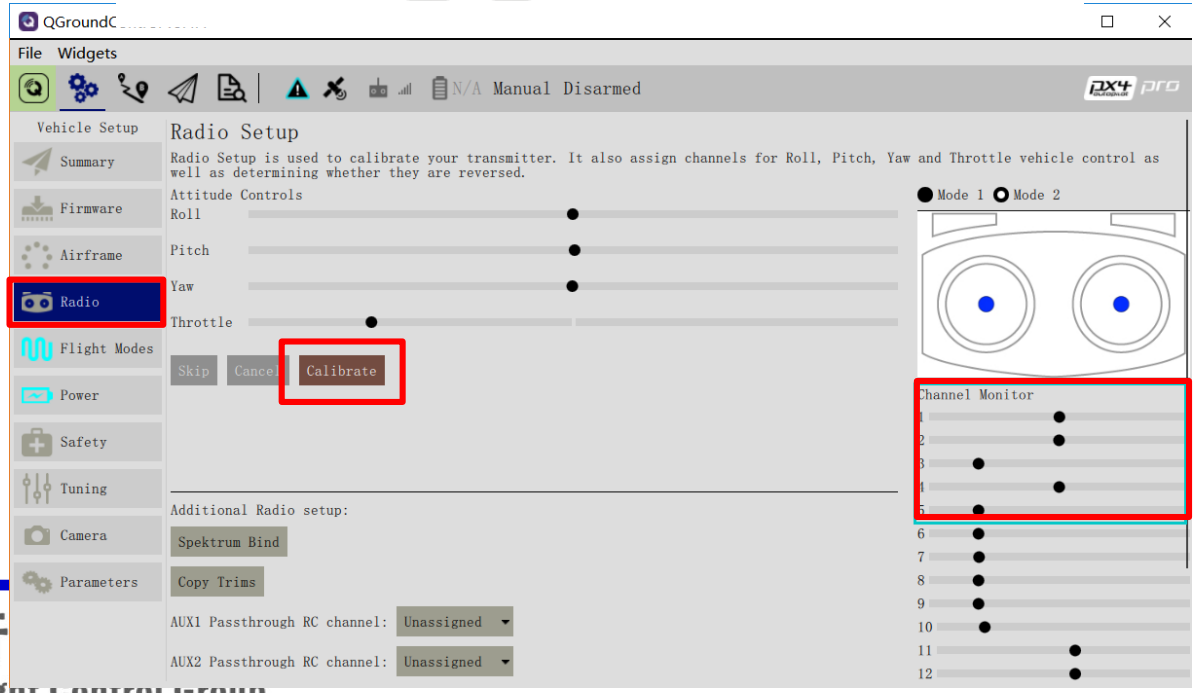
1.7 遥控器配置与校准

1、正确连接自驾仪与接收机，用USB数据线连接自驾仪与电脑，打开遥控器，打开QGroundControl地面站软件，点击右下图所示的“Radio”标签页。

2、依次从左到右（或从上到下）拨动遥控器的CH1到CH5通道（见右上图），观测右下图地面站右侧红框区域中各个通道的白点。如果观测到：1、2、4、5、6号白点从左到右移动（PWM从1100到1900）；3号白点从右向左移动，说明遥控器设置正确。否则需要重新配置遥控器。

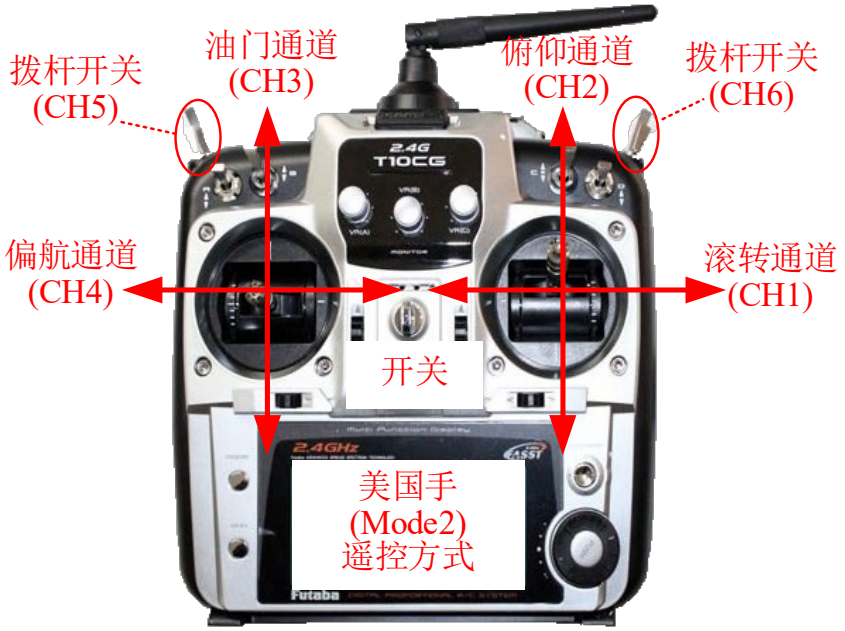
3、点击右下图的“Calibrate”按钮，按提示可以校准遥控器。



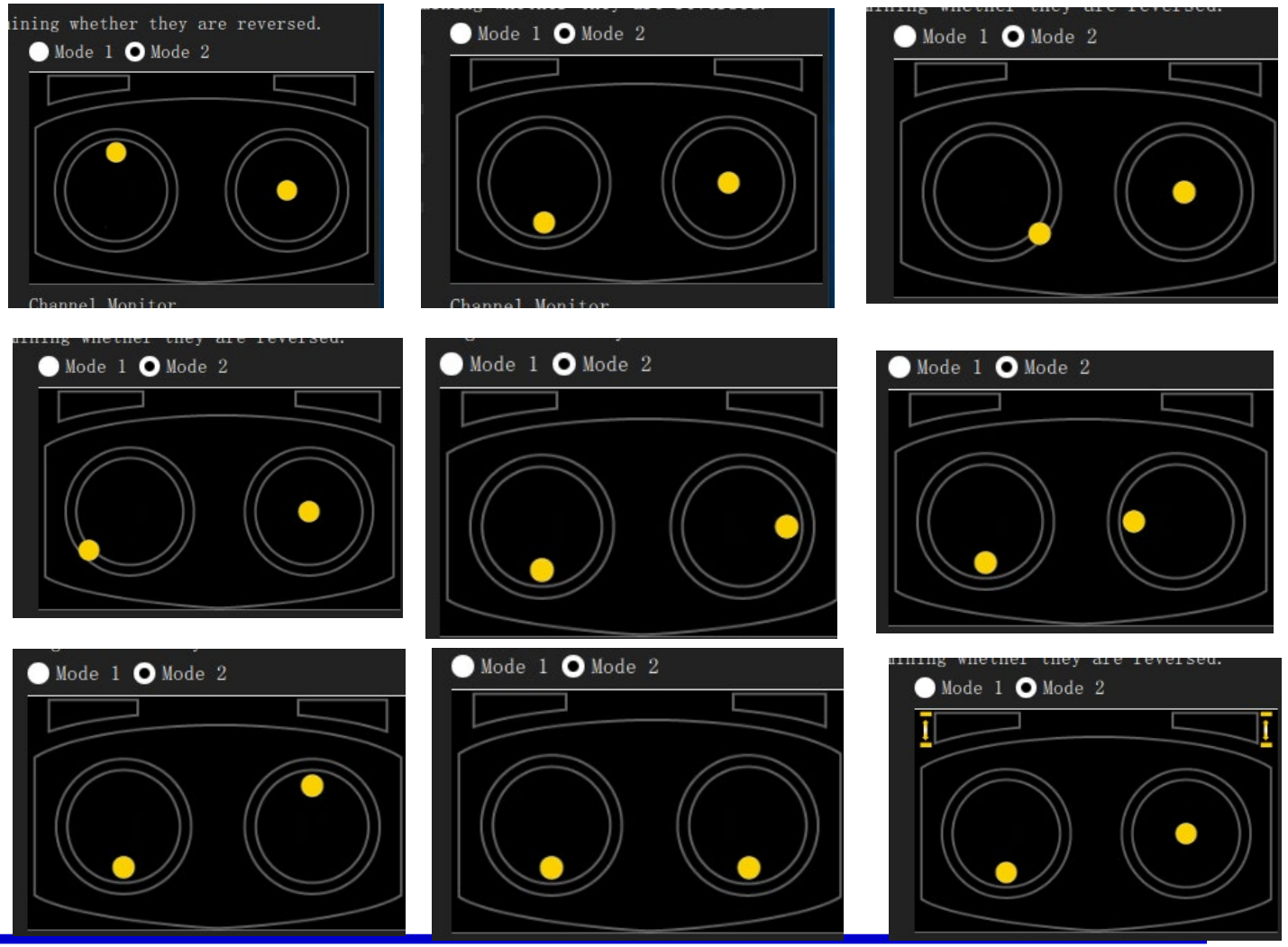


1.课程学习

4、点击QGC地面站上的“Calibrate”–“Next”按钮，然后依次将摇杆置于右图所示位置（根据QGC页面的实时提示）即可完成遥控器校准。



油门：控制上下运动，对应固定翼油门杆
偏航：控制机头转向，对应固定翼方向舵
俯仰：控制前后运动，对应固定翼升降舵
滚转：控制左右运动，对应固定翼副翼

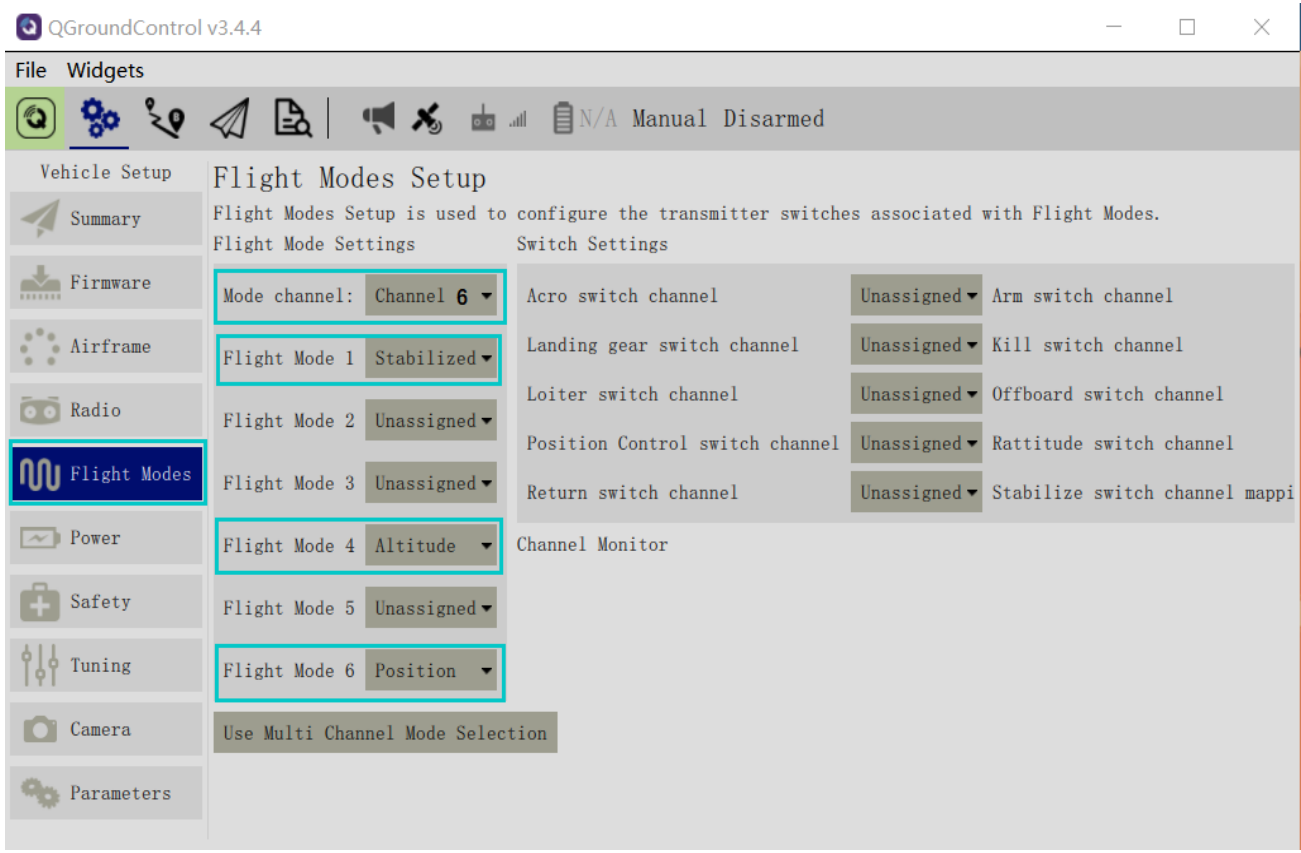


1.课程学习

1.8 飞行模式设置

1、经过上面的遥控器校准步骤后，点击地面站进入“Flight Modes”（飞行模式）设置页面，选择“Mode Channel”（模式通道）为前面测试过的CH6通道。由于CH6通道是一个三段开关，开关的顶部、中部、下部档位分别对应了“Flight Mode 1、4、6”三个标签。

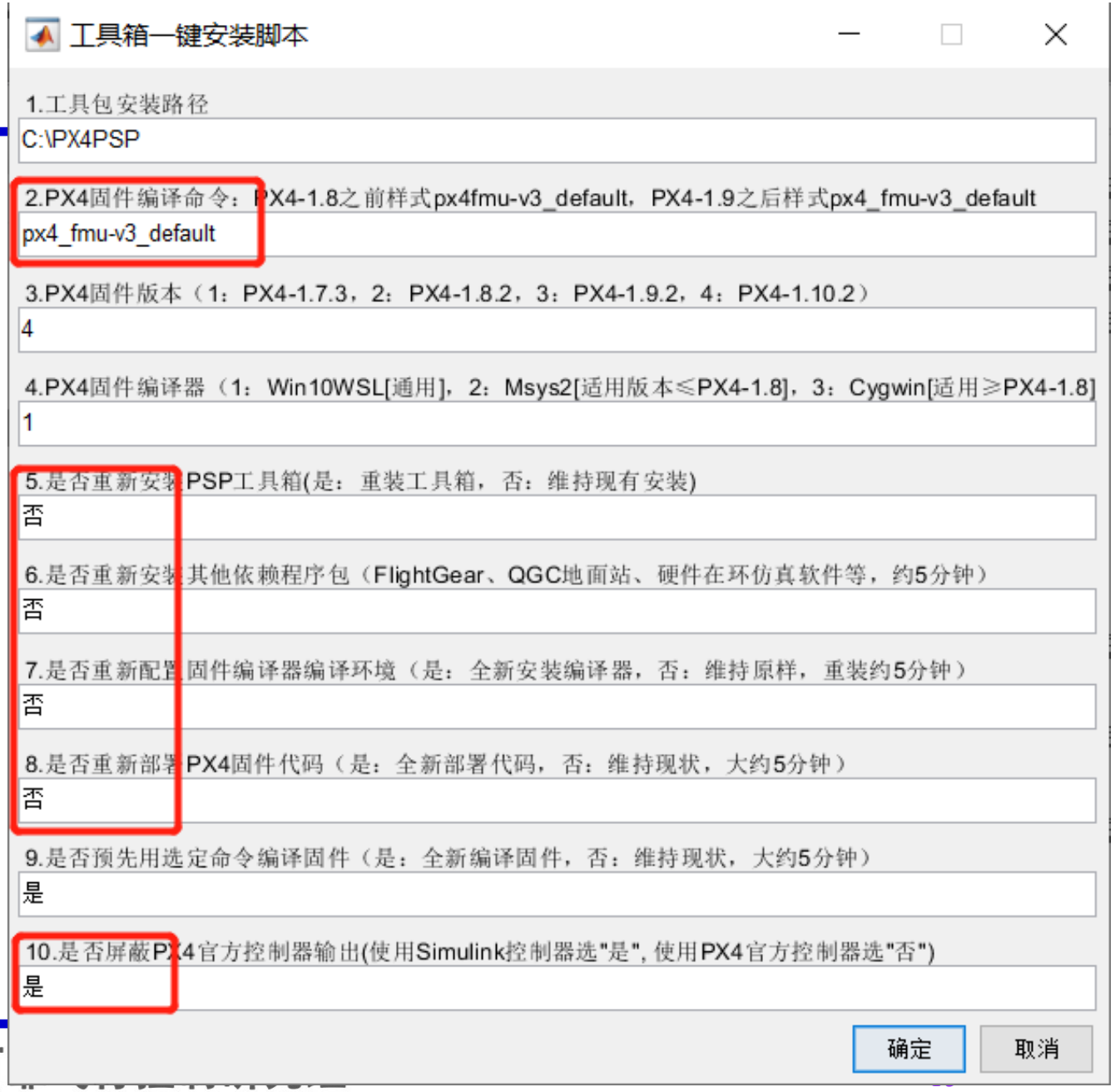
2、如右图所示，将这三个标签分别设置为“Stabilized”（自稳模式，只有姿态控制）、“Altitude”（定高模式，姿态和高度控制）和“Position”（定点模式，有姿态、定高和水平位置控制）。在后续的处理器在环仿真中，可以通过切换不同的模式来体验不同的控制效果。



1.课程学习

1.9 更改开发模式配置

- 从视觉/集群模式切换到底层飞控开发模式，或者反向切换，只需要重新运行“OnekeyScript.p”脚本，再配置依次即可
- 不过为了节省安装时间，如右图所示，一些已经安装过的组件可以不用安装，选择“否”即可。
- 如果只是修改固件编译命令，例如从“px4fmu-v3_default”切换到“px4fmu-v5_default”，只需要在MATLAB命令行窗口中输入“PX4CMD px4fmu-v5_default”即可，不需要重新运行安装脚本。



北航可
BUAA Reliable Flight Control Group



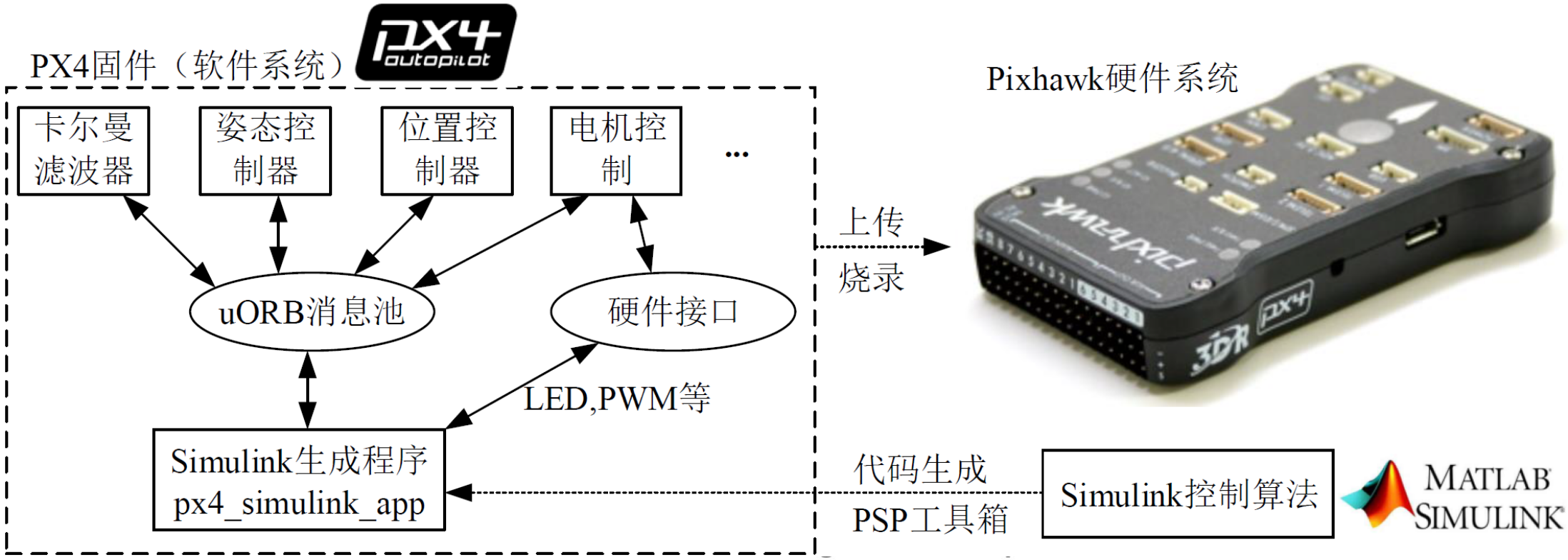
大纲

1. 课程学习
2. 原理详解
3. 进阶例子
4. 小结

2.原理详解

2.1 Pixhawk/PX4/Simulink代码生成平台构架

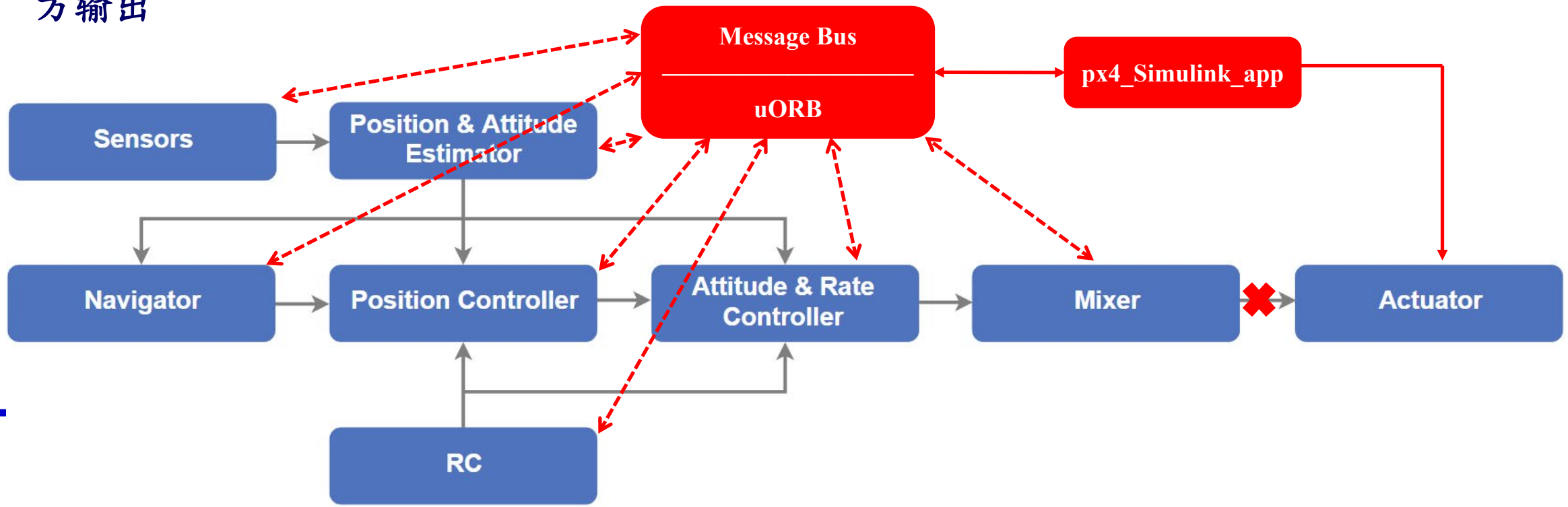
- Pixhawk是硬件（相当于电脑主机），PX4是飞控软件（相当于Windows操作系统），Simulink控制器生成代码后编译成固件（相当于系统iso镜像），上传到Pixhawk硬件（相当于重装系统），Simulink控制器以新线程（相当于电脑上第三方APP）独立于PX4官方控制器（相当于系统预装软件），并行运行



2.原理详解

2.2 为何屏蔽PX4输出

- PX4采用uORB发布与订阅消息机制，任何APP都可以从uORB消息池中获取和发布数据
- Simulink代码生成到Pixhawk后生成名为px4_Simulink_app的一个APP，可以通过uORB消息池机制与其他APP通信
- Simulink_App不能和PX4控制器同时访问电机，不然会产生冲突，因此需要屏蔽PX4官方输出



2.原理详解

2.3 用Simulink控制器替换PX4官方滤波器、混控器等APP的方法

- 生成的Simulink代码也可以用于替换如右图所示PX4控制软件的某些原生模块（传感器、滤波器、姿态控制器等），但是需要手动修改PX4固件代码来屏蔽原始模块的输出接口。例如，想用Simulink实现一个滤波器模块（输入传感器数据，输出状态滤波数据）来替换PX4原始的滤波器，需要手动屏蔽右图中的“Position & Attitude Estimator”滤波器模块向uORB消息池发布滤波后姿态数据（对应名为vehicle_attitude的uORB消息）。具体操作示例为：
- 打开“Firmware\src\modules\ekf2\ekf2_main.cpp”文件（扩展卡尔曼滤波器模块对应代码）中；
- 在其中屏蔽掉“ORB_ID(vehicle_attitude)”消息相关的发送代码。例如，搜索关键字“_att_pub”的代码行，并找到其中含有“publish”和“att”的发送代码行，将其替换为“UNUSED(att);”。这里UNUSED用于防止编译器出现变量未使用的警告。
- 在Simulink中编写姿态滤波器，并用uORB Write模块发送vehicle_attitude消息即可实现姿态滤波器功能的替换。

注：另一种可行方法是修改PX4模块启动脚本文件"Firmware\ROMFS\px4fmu_common\init.d\rcS"，并注释掉想要屏蔽的模块

2.原理详解

2.4 Simulink自动代码生成配置

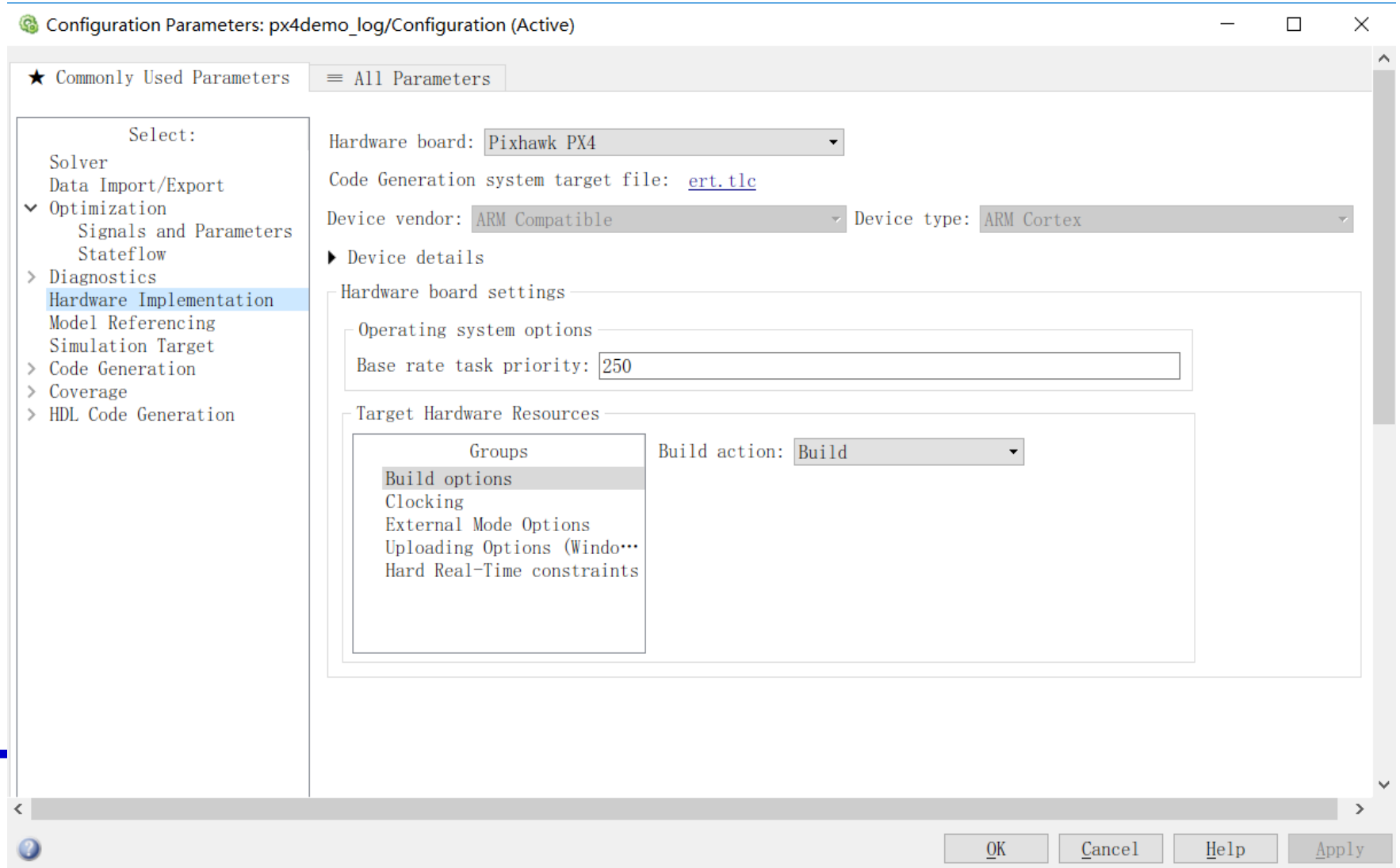
打开任意slx例程文件

1、进入Simulink设置页面

2、选择Hardware board设置后，会自动完成所有代码生成设置

3、可以定制任务优先级

4、配置编译选项



2.原理详解

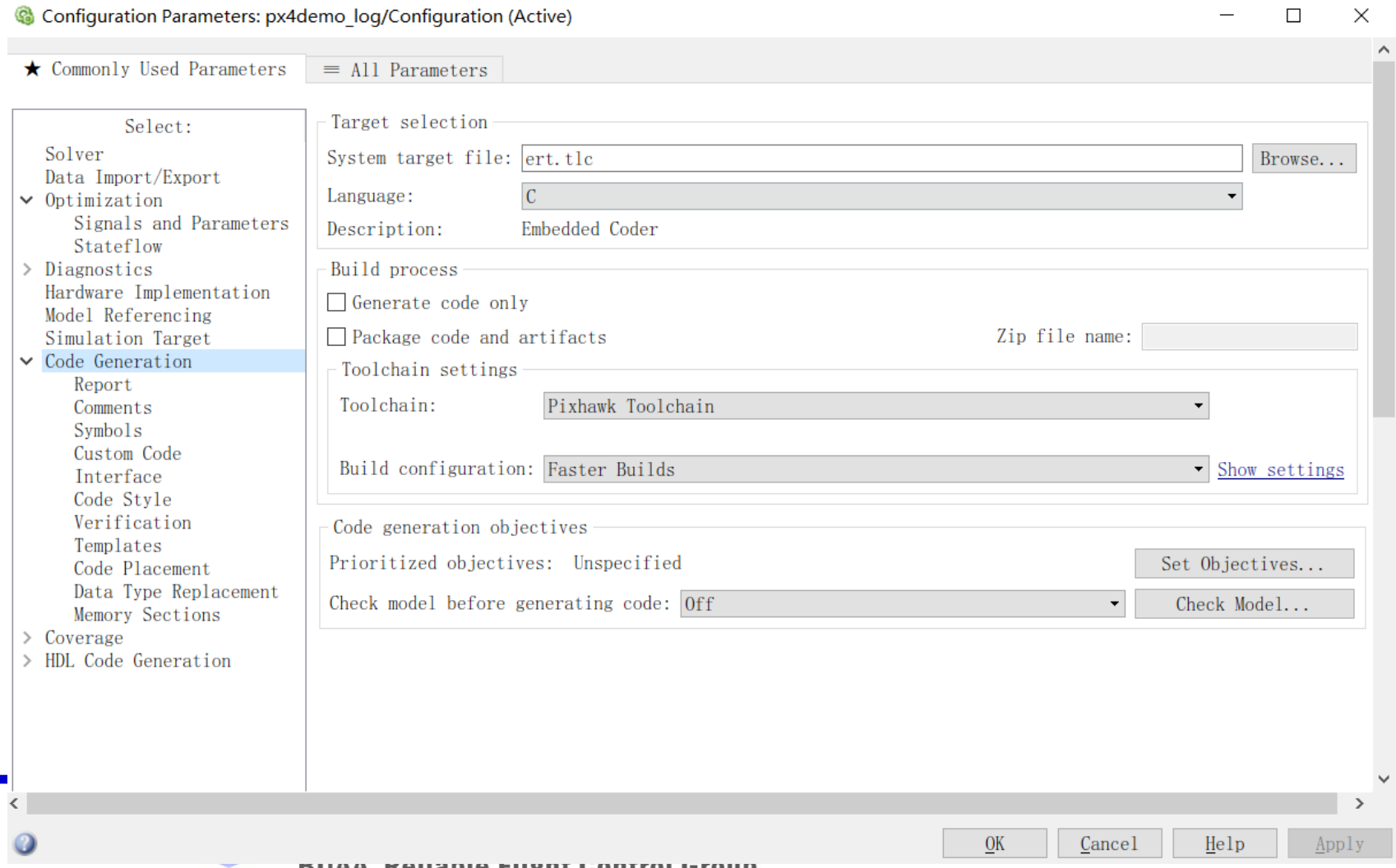
2.4 Simulink自动代码生成配置

代码生成的配置主要在Code Generation页面

1、System Target File对应生成代码的运行平台，也就是代码模板

2、Language对应生成的语言，可选C或C++

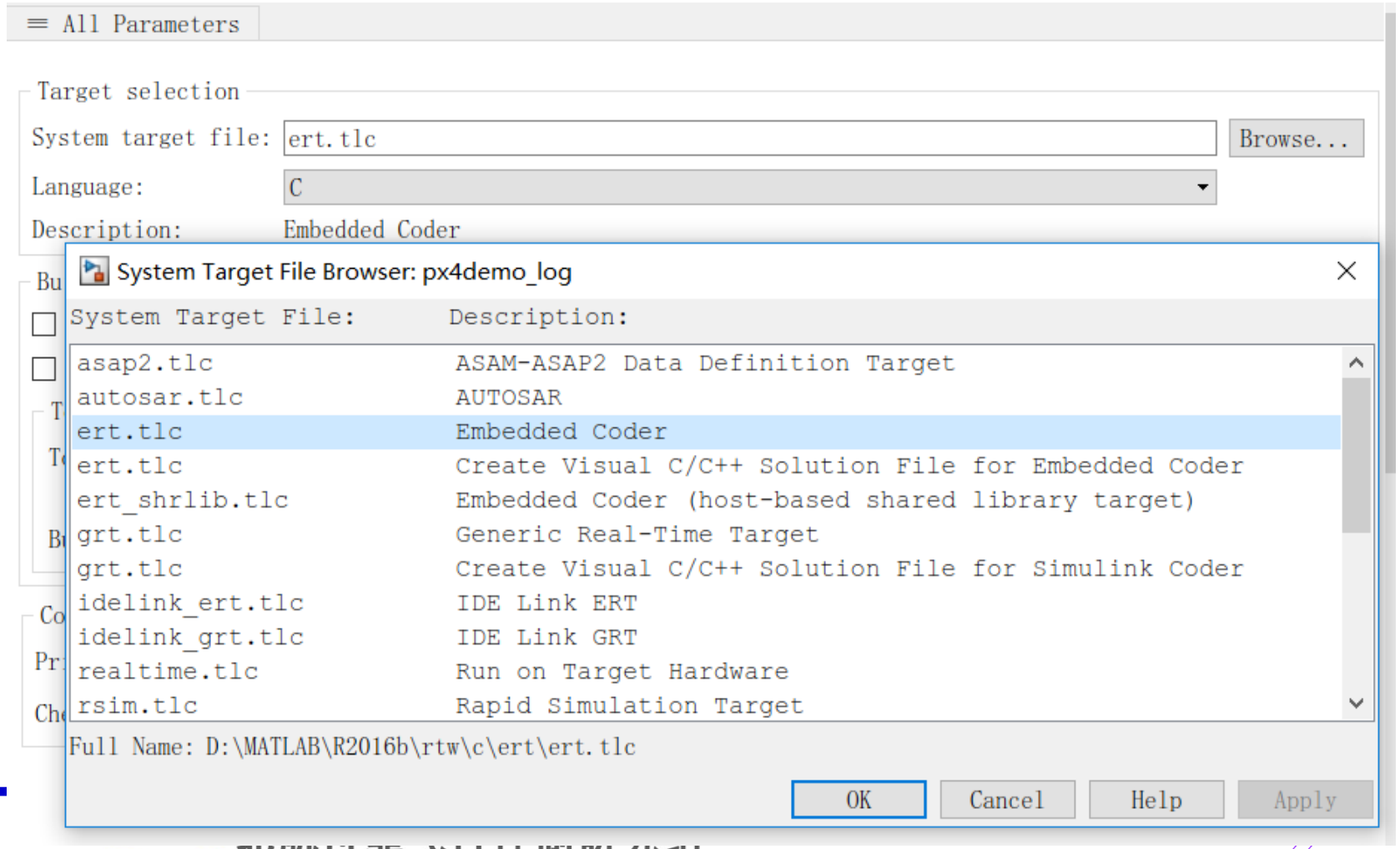
3、还有一些编译器设置选项



2.原理详解

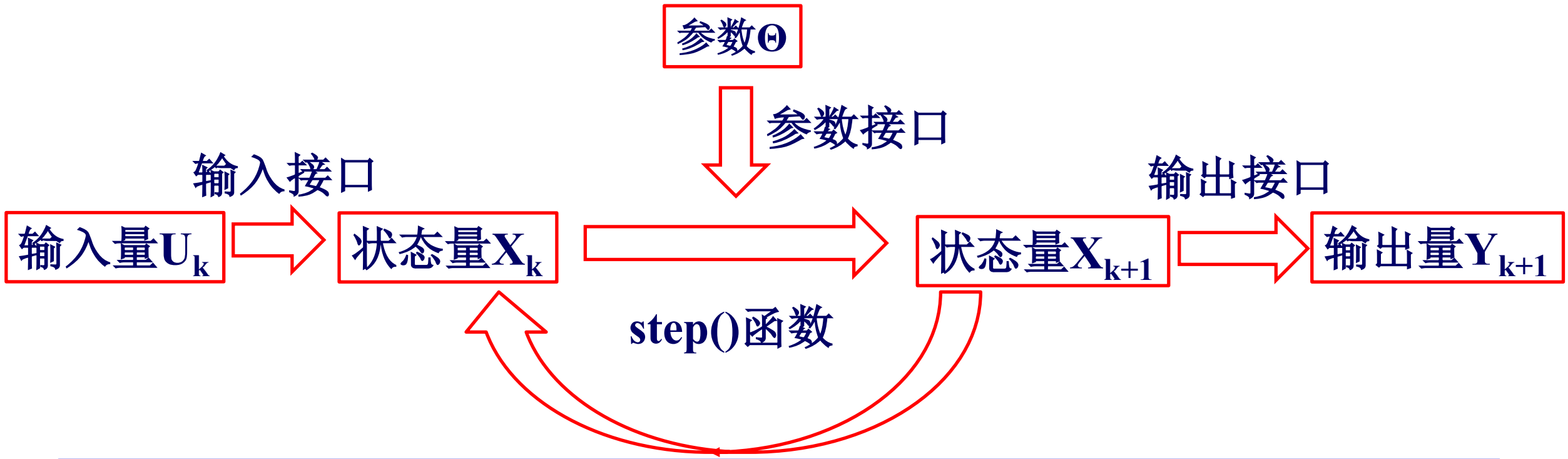
2.4 Simulink自动代码生成配置

- ert.tlc是最常用的一种代码生成方式
- 主程序最终为一个step()函数
- 需要自己在嵌入式系统中利用中断或定时器，按设定步长调用
- 例如：仿真步长0.001s，则嵌入式中断之相同



2.4 Simulink自动代码生成配置

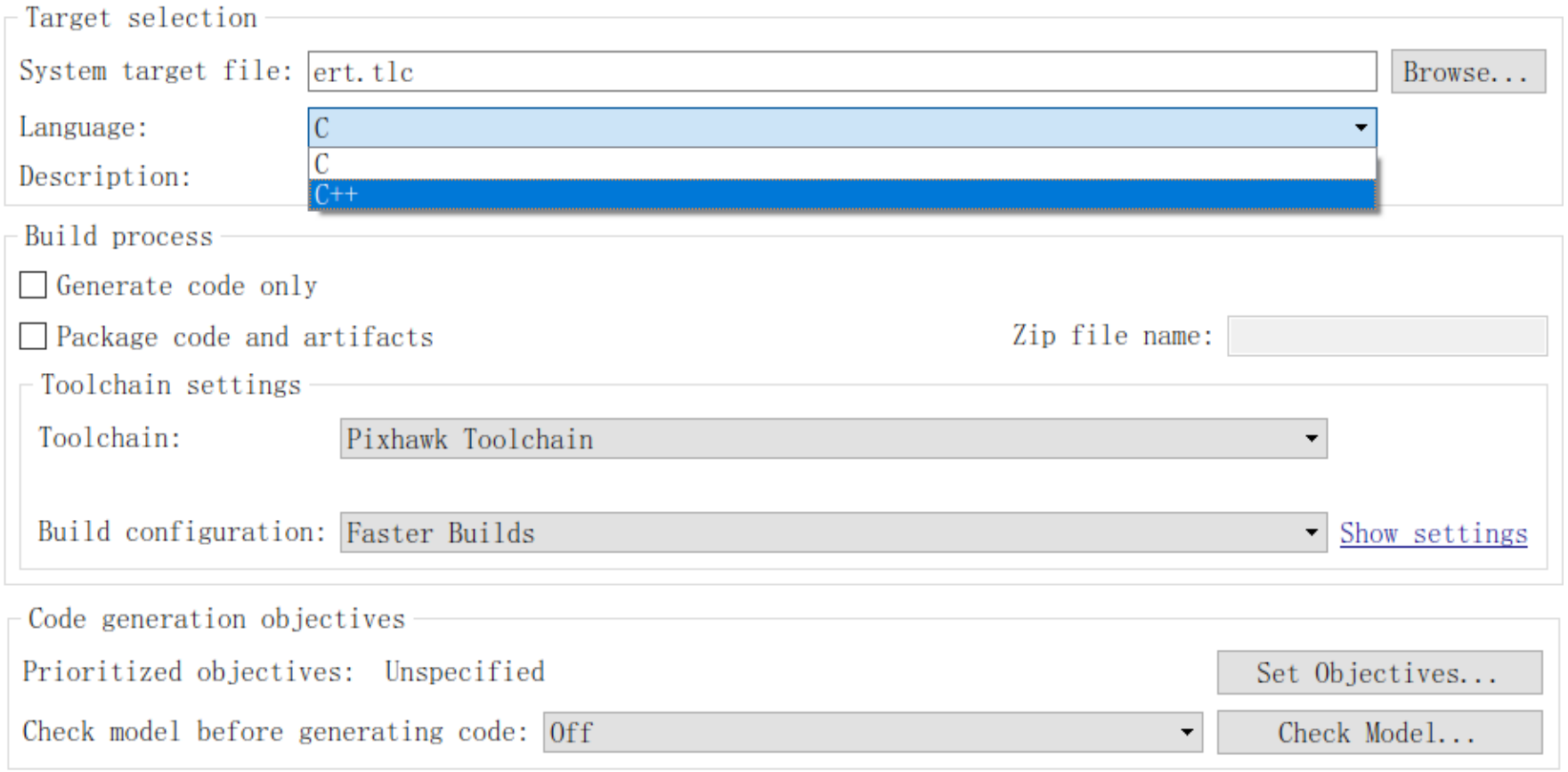
ert.tlc生成的嵌入式系统运行示意图。step()函数可选龙格库塔法、欧拉法等近似积分方法；参数接口允许实时改变模型参数；输入输出接口允许其他程序调用。



2.原理详解

2.4 Simulink自动代码生成配置

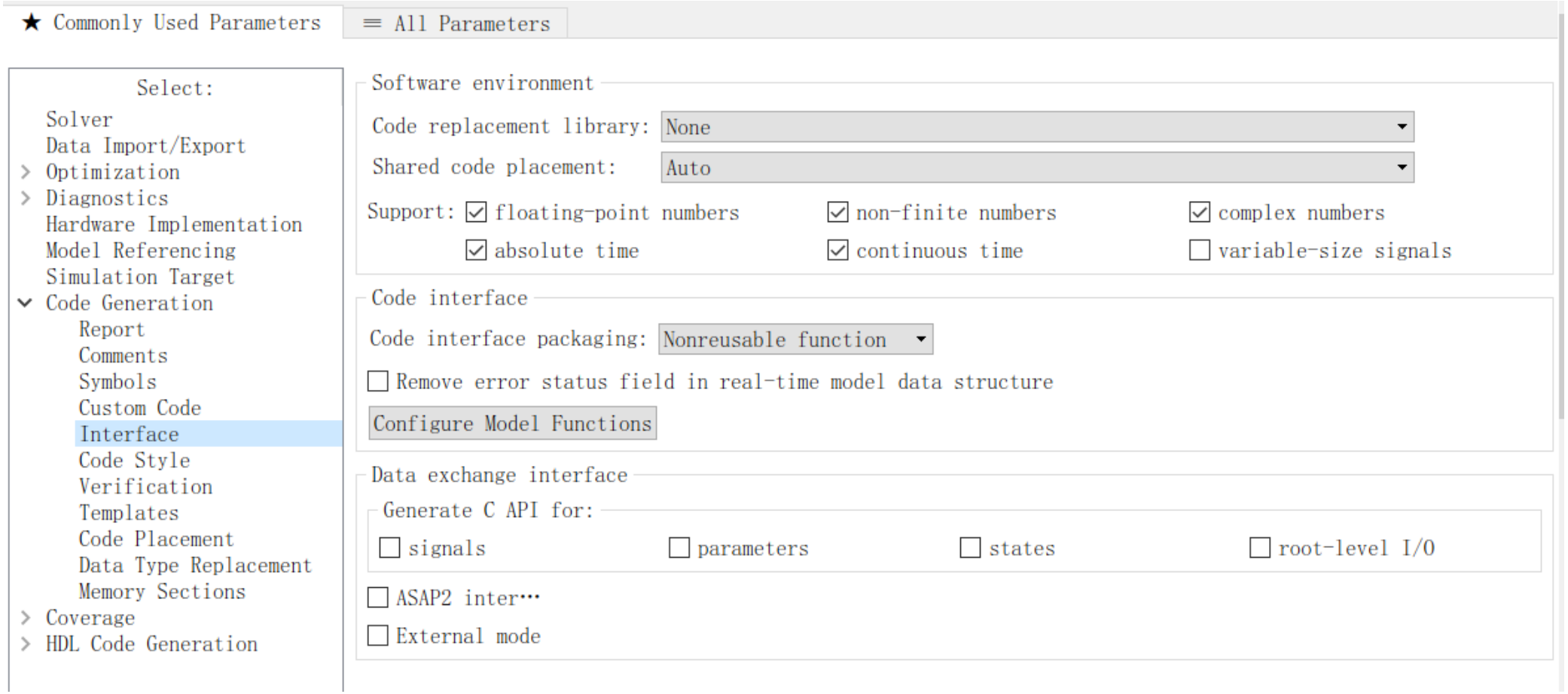
- C/C++的区别
- C生成代码更为简单，但是扩展性较弱
- C++可以将整个程序封装为一个类
- 便于后期的继承与扩展



2.原理详解

2.4 Simulink自动代码生成配置

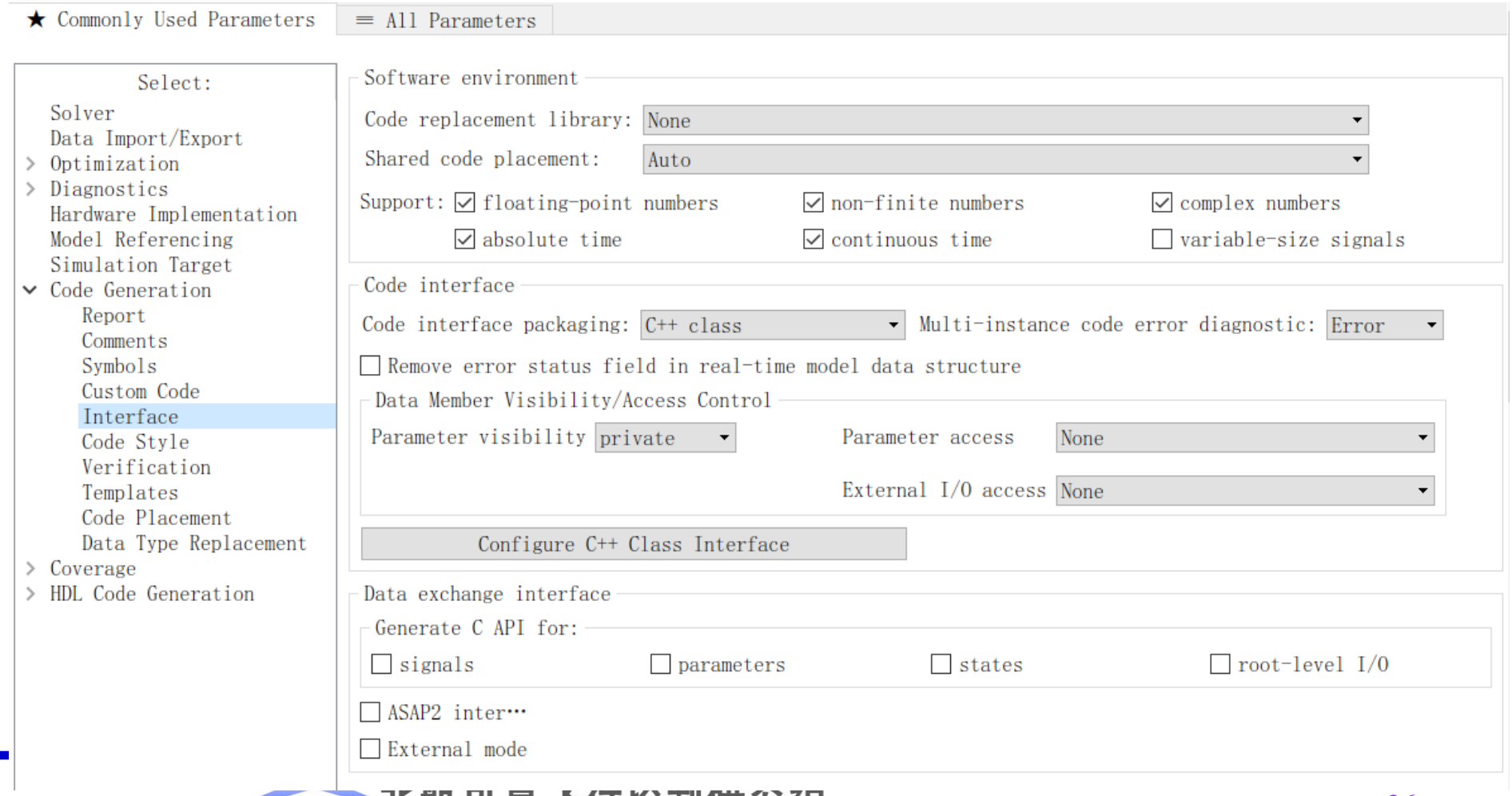
- C对应的Interface页面
- 可以设置一些仿真方式，例如，是否支持复数，是否支持连续时间
- 也可以定义输出接口



2.原理详解

2.4 Simulink自动代码生成配置

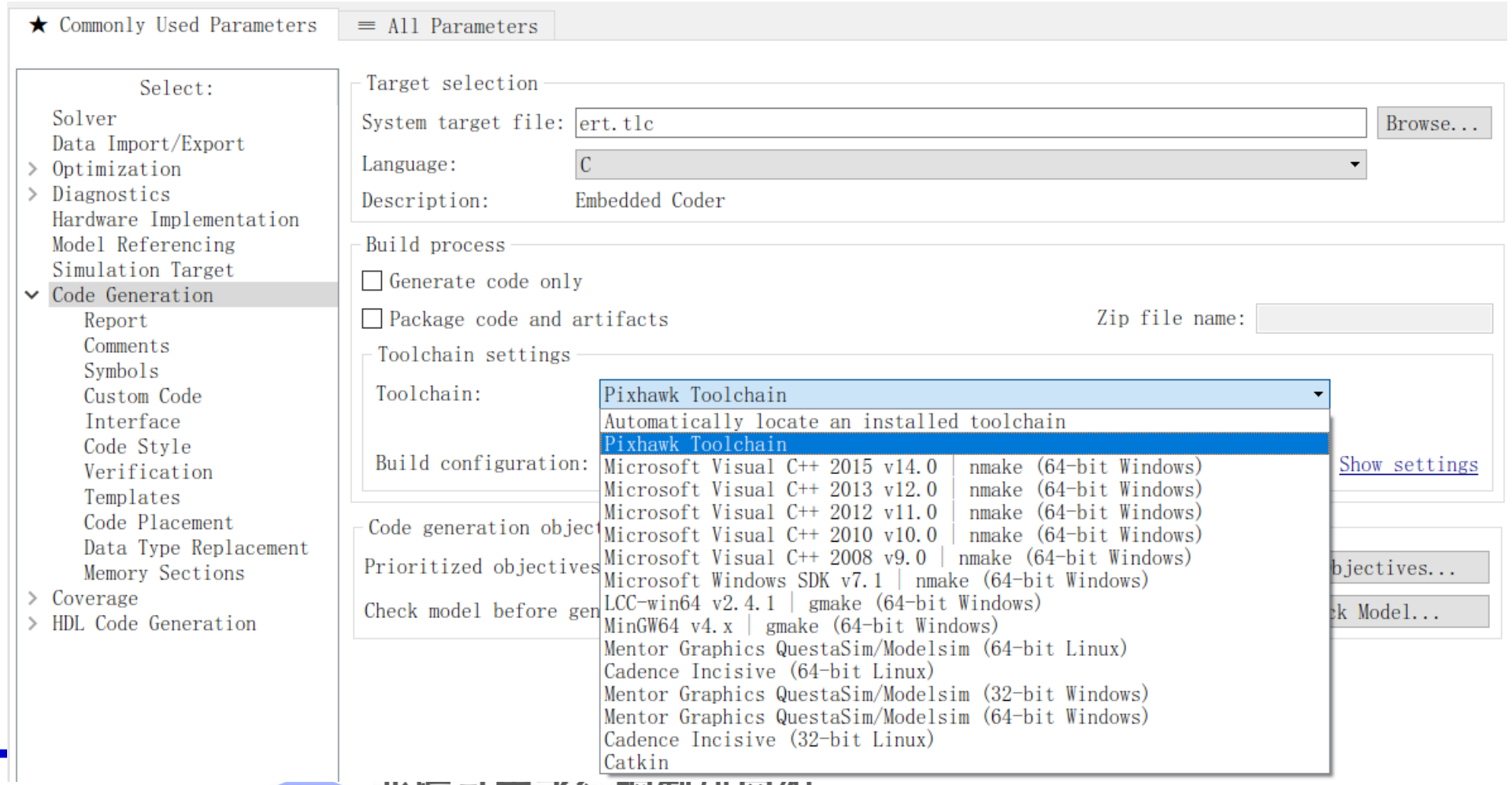
- C++对应的Interface页面
- 还可以设置参数结构体是否共有
- 是否支持多实例
- 还有是否生成各种外部接口



2.原理详解

2.4 Simulink自动代码生成配置

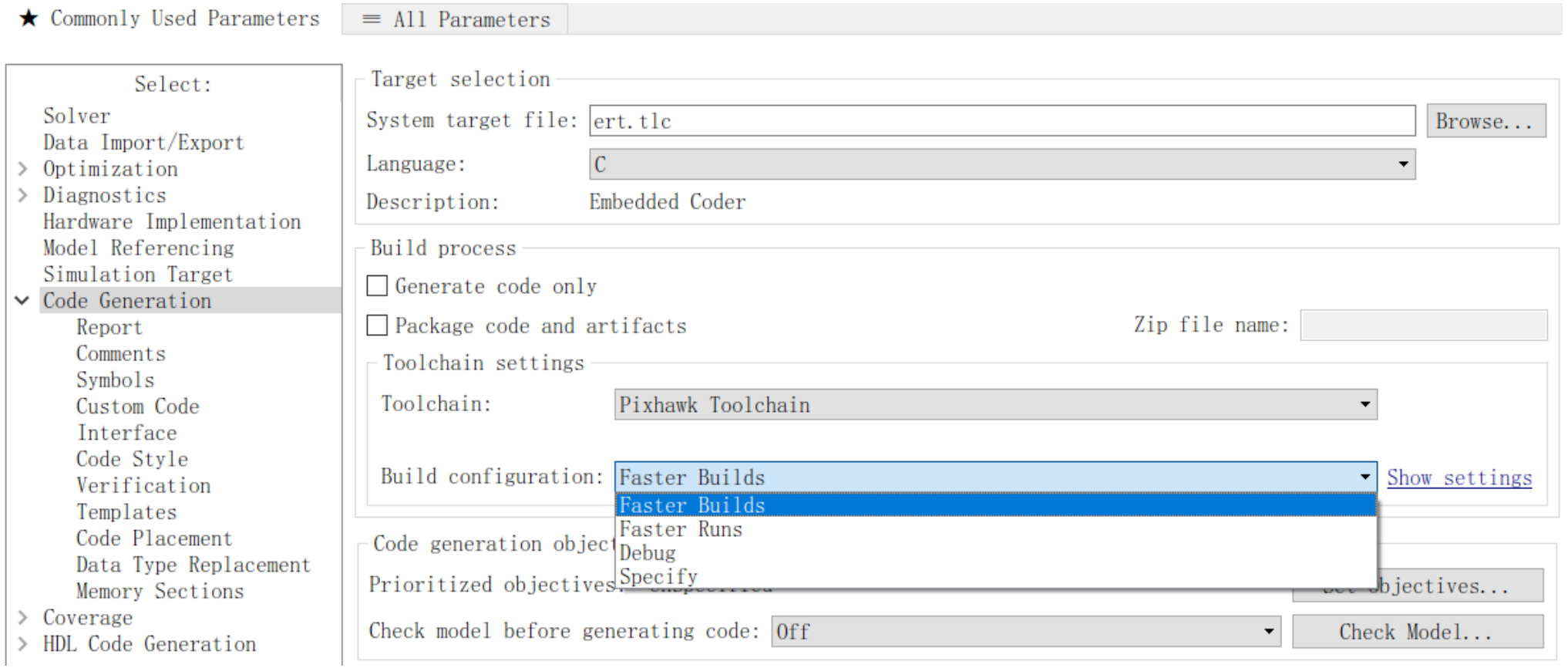
- 选择目标编译工具
- 这里选择Pixhawk Toolchain
- 当然也可以选择Visual C++
- 或者其他的编译器



2.原理详解

2.4 Simulink自动代码生成配置

- 选择编译配置
- **Faster Builds** 得到的代码量较小，可以使编译更快
- **Faster Runs** 会优化代码和编译保证运行效率更快





大纲

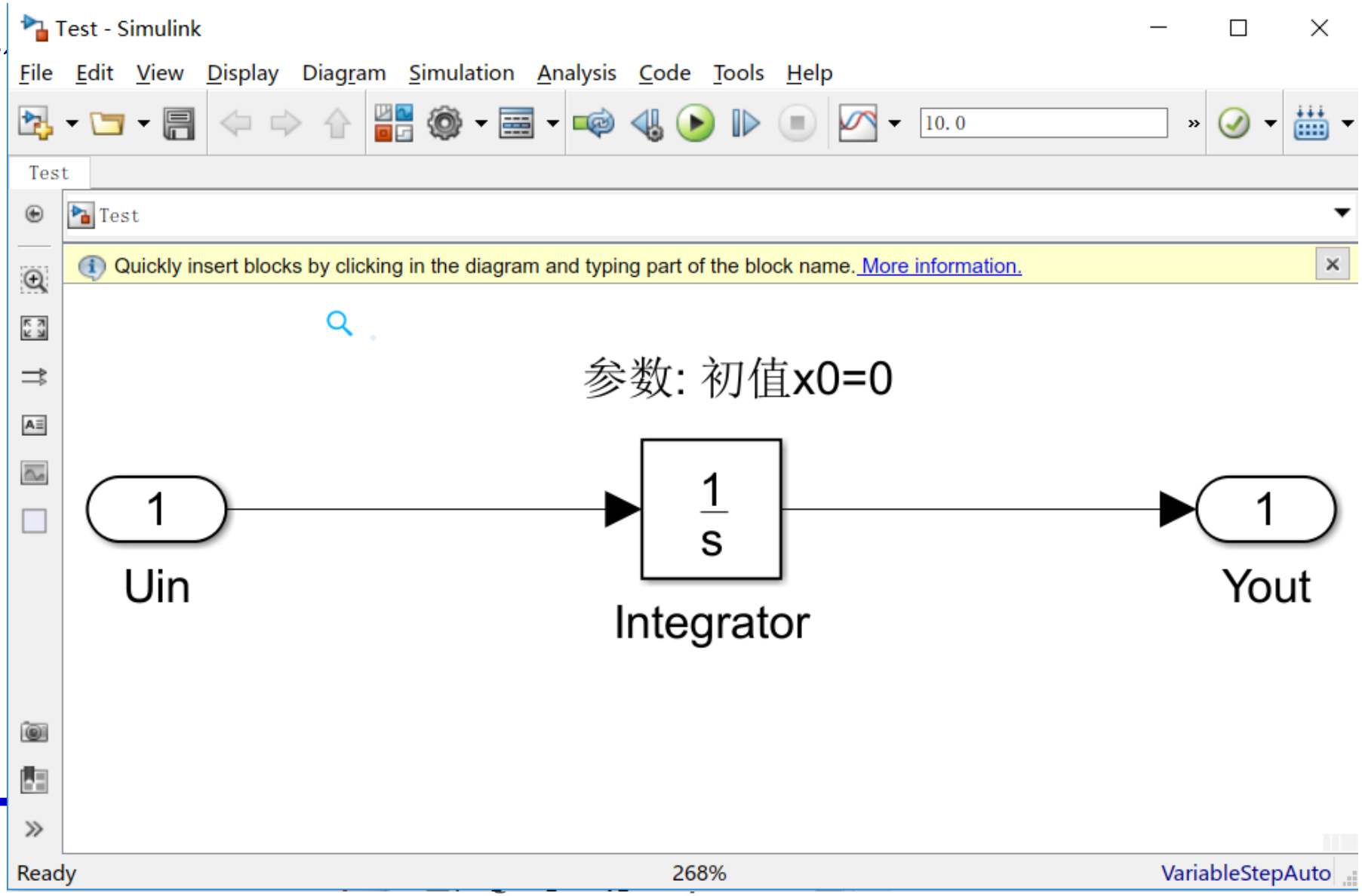
1. 课程学习
2. 原理详解
3. 进阶例子
4. 小结

3.进阶例子

注：本例子需要用到VS编译器，请按照“RflySim高级版_第01讲_总体介绍”中1.8节的步骤安装并配置

3.1 自主生成C/C++

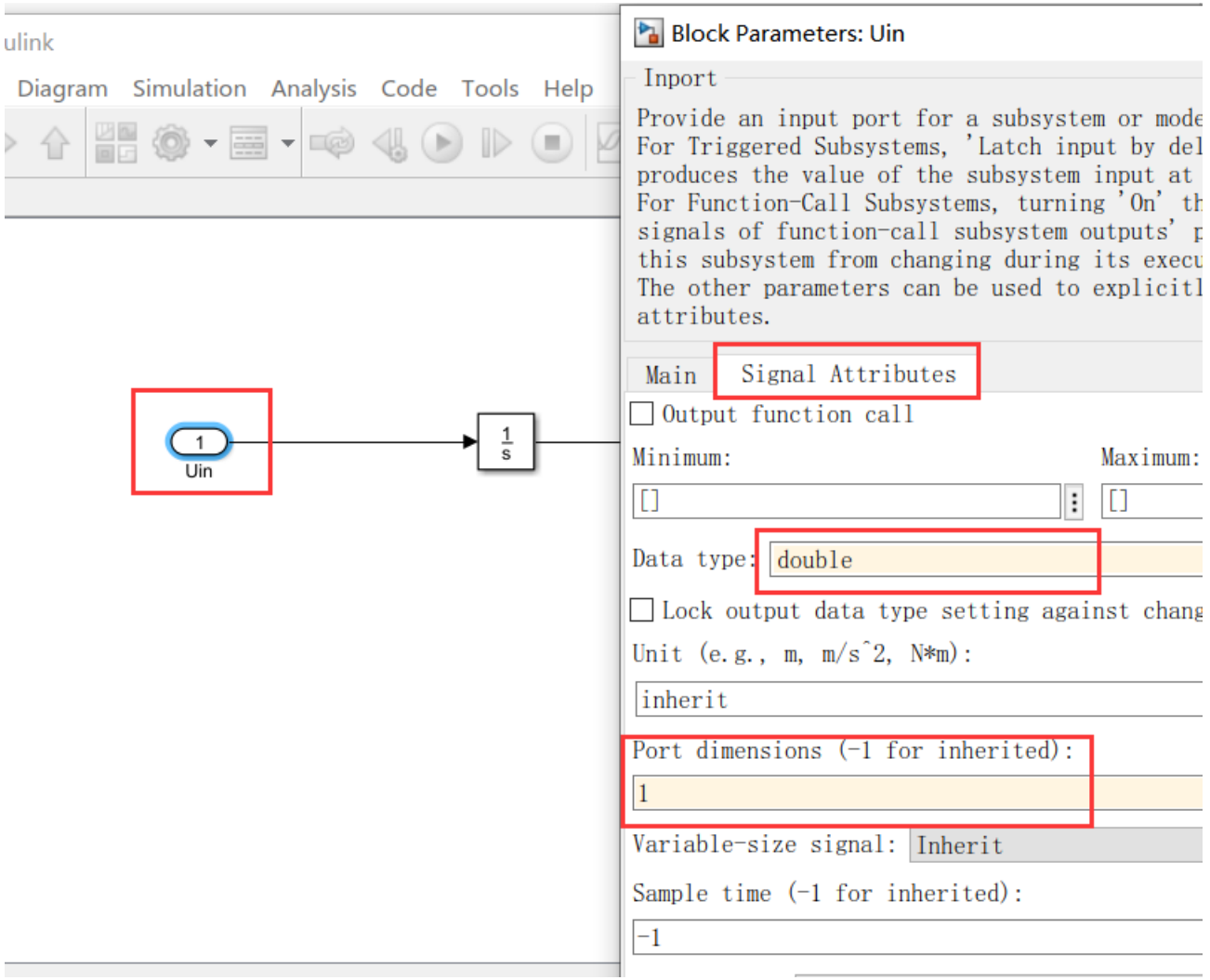
- 按右图搭建Simulink模型
- 输入命名为Uin
- 输出命名为Yout
- 积分的初值定义为X0
- 上述变量的名字需要记住，它们对应了生成C++代码的变量名
- 例程见“eA\Sec2\CodeGen Example.slx”



3.进阶例子

3.1 自主生成C/C++代码例子

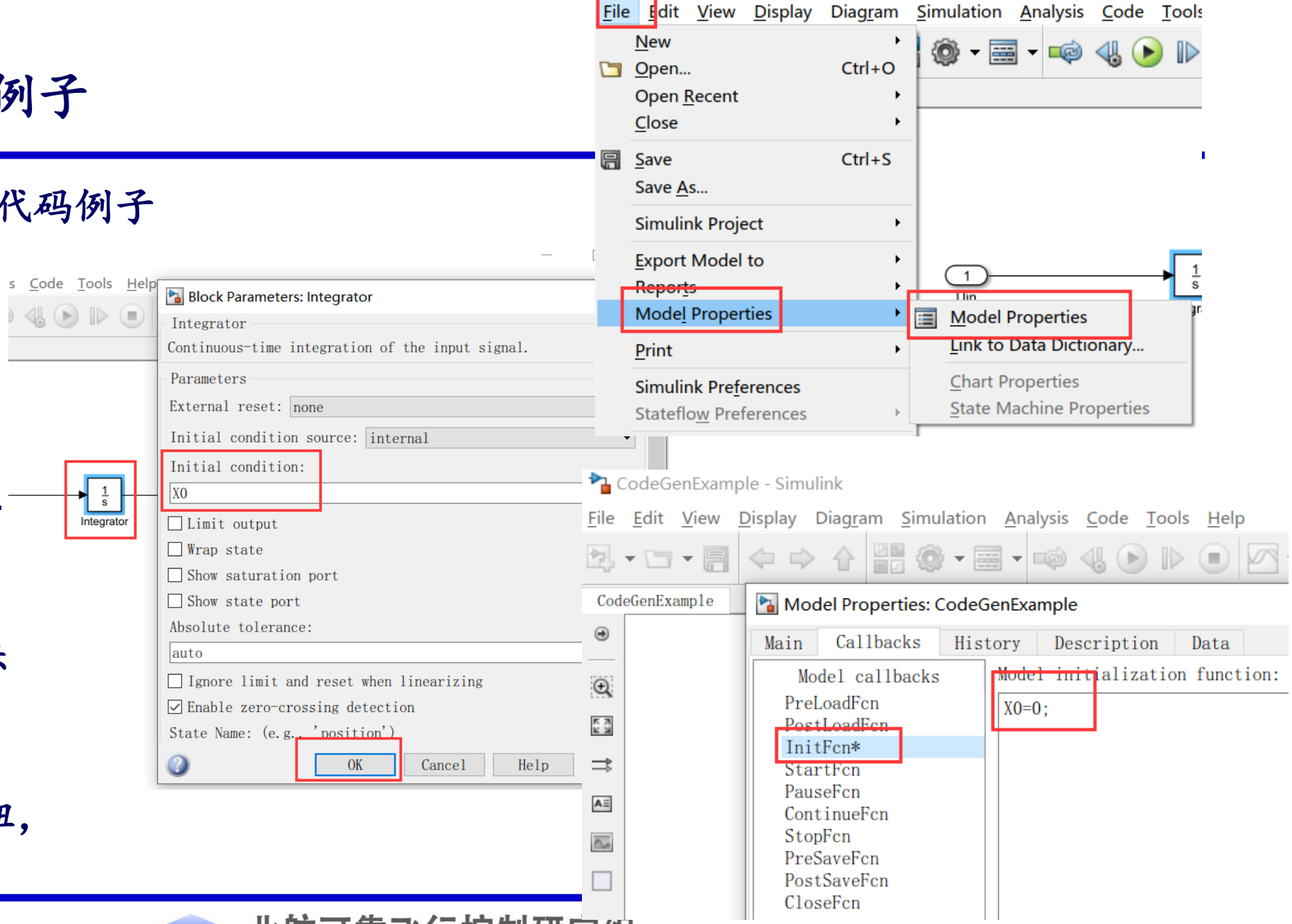
- 双击Uin图标，进入参数设置页面
- 进入Signal Attributes页面
- 设置数据类型“Data Type”为“double”
- 设置数据维度“Port dimensions”为“1”
- 这样我们就定义了，代码生成后输入接口的数据格式。
- 同理设置“Uout”输出接口



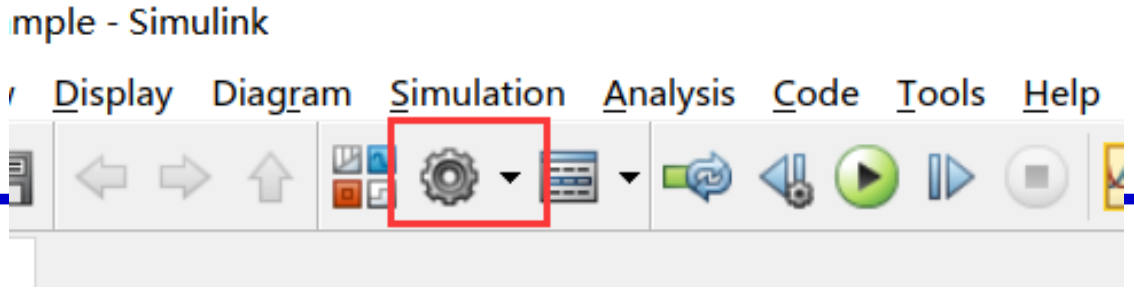
3.进阶例子

3.1 自主生成C/C++代码例子

- 双击积分模块，进入参数设置页面。
- 设置一个带名字的参数“X0”
- 打开Simulink-菜单栏-File-Model-Property-Model-Property页面
- 在Callbacks-InitFcn标签页加入初始化脚本“X0=0”
- 点击Simulink运行按钮，看能否正确运行。

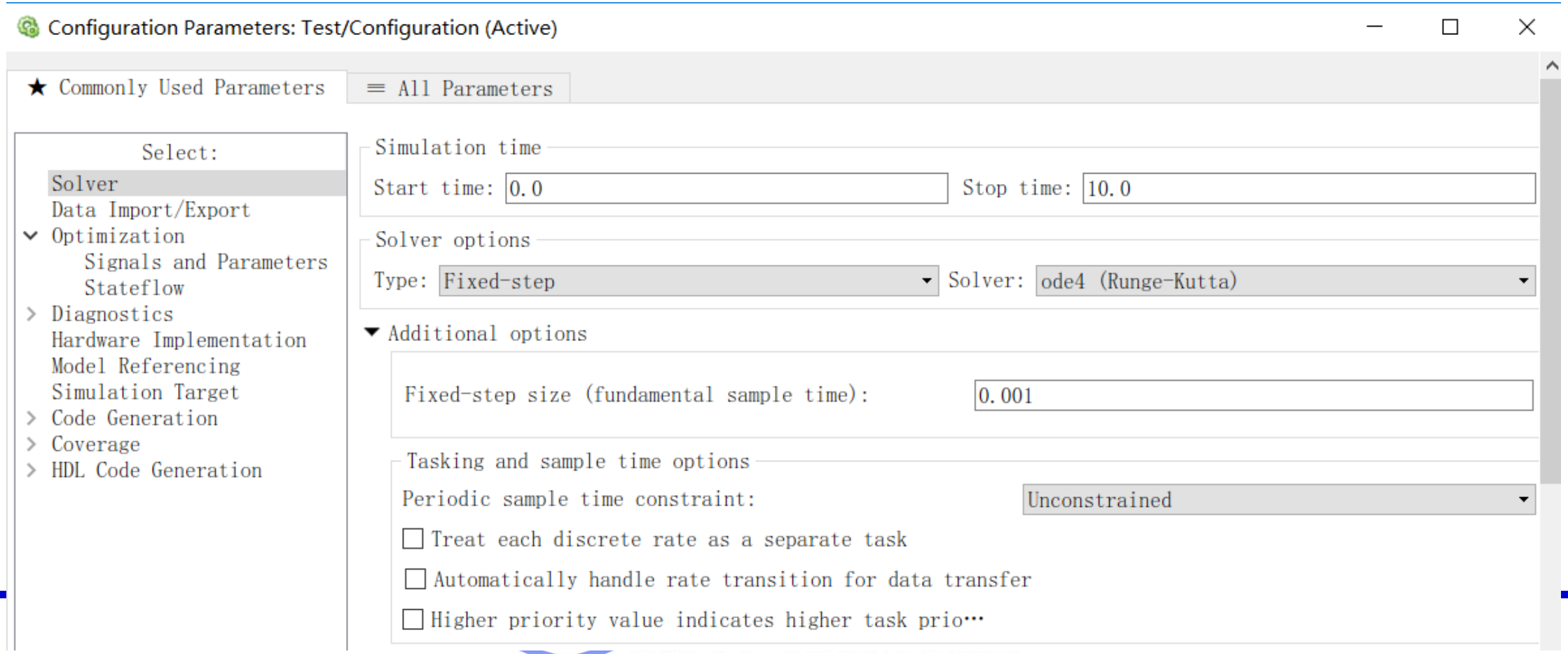


3.进阶例子



3.1 自主生成C/C++代码例子

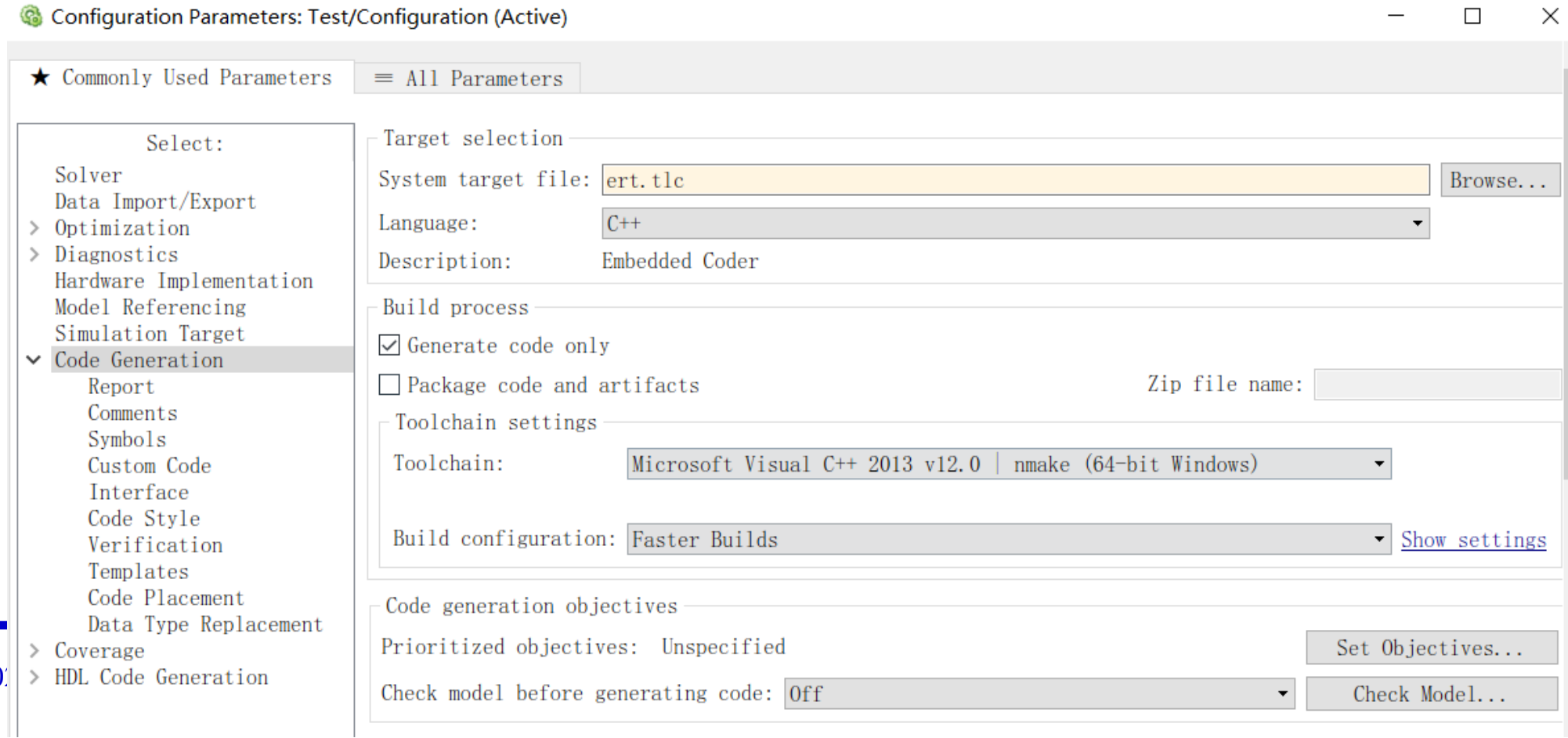
- 打开设置页面，设置仿真为定步长，四阶龙格库塔法求解器，步长为0.001s



3.进阶例子

3.1 自主生成C/C++代码例子

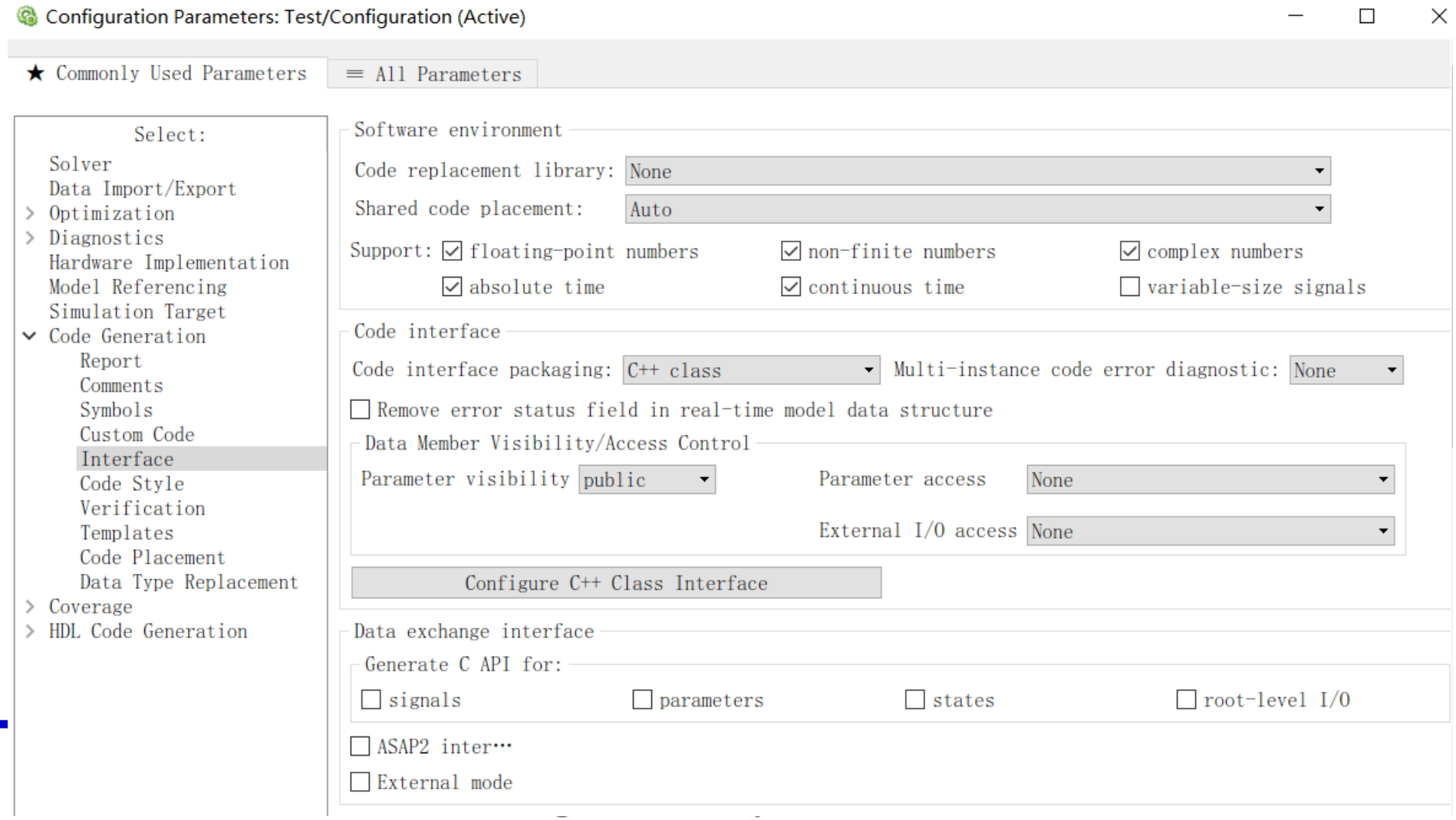
- 代码生成方式选择ert.tlc，可用于windows，Linux和各类嵌入式平台；语言选择C++，便于通过继承方式调用生成代码；工具选择Visual Studio C++



3.进阶例子

3.1 自主生成C/C++代码例子

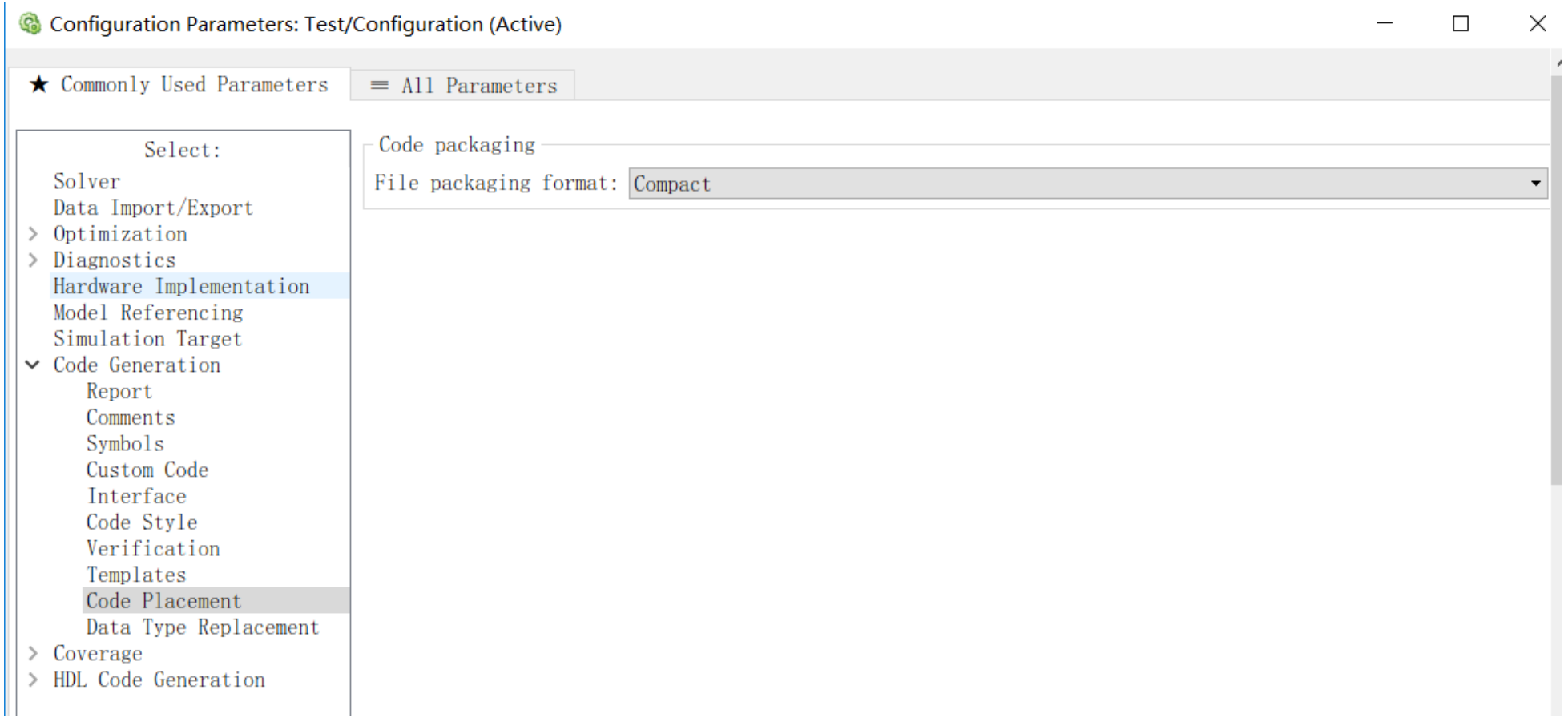
- 因为包含连续模块（积分模块）因此需要勾选continuous time，不然编译报错。
- 此外将参数可见性设为public，是的参数结构体为共有变量，便于访问



3.进阶例子

3.1 自主生成C/C++代码例子

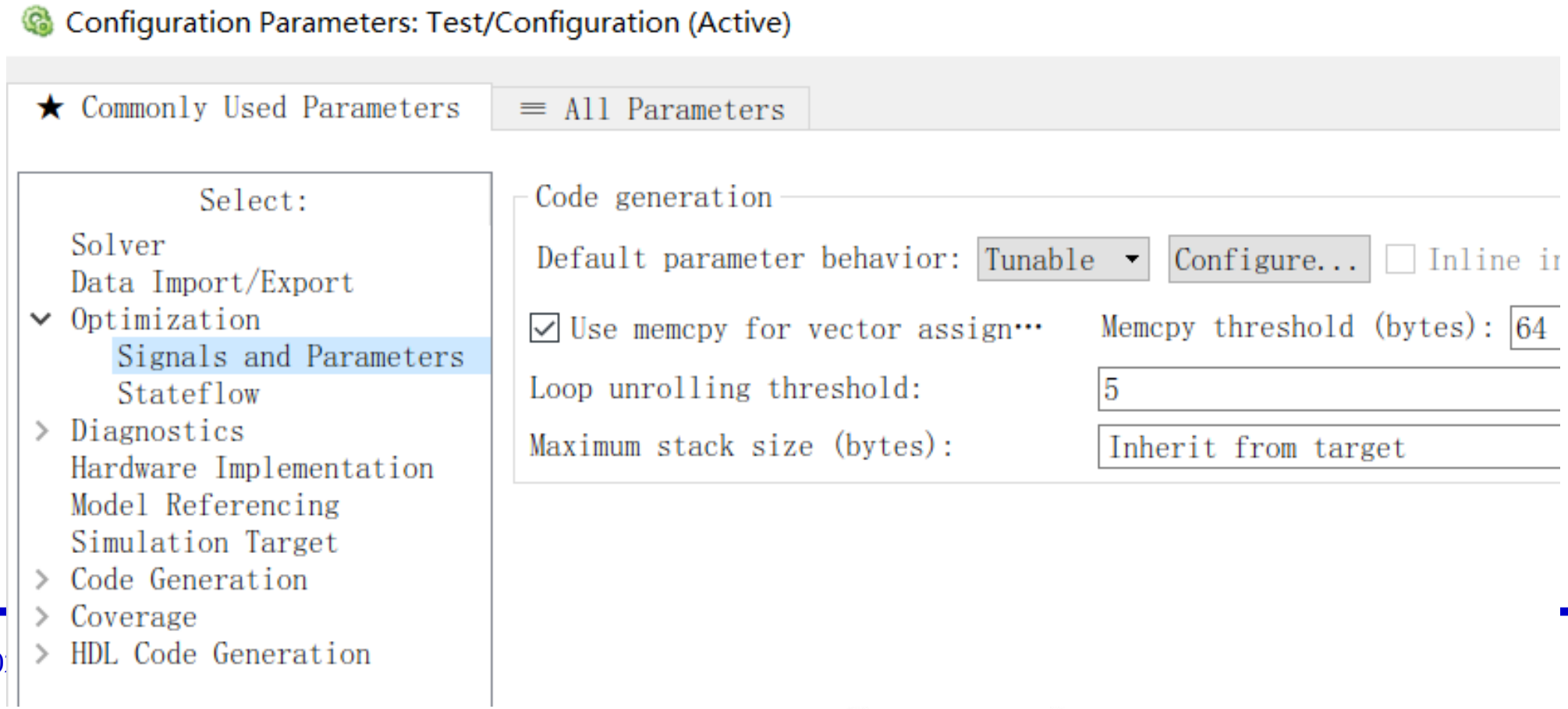
在Code placement页设置文件打包类型为compact，尽量避免生成多余文件，使得代码的可读性最强



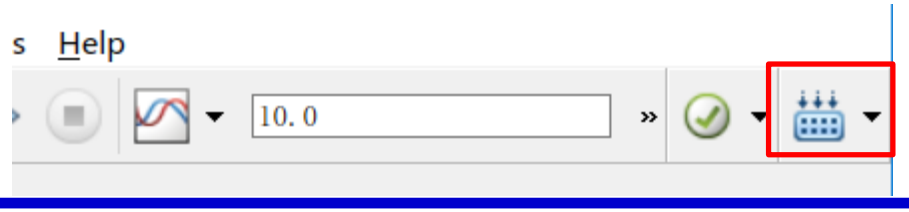
3.进阶例子

3.1 自主生成C/C++代码例子

- 设置参数为Tunable是的我们可以运行时修改参数。注：inline形式更省内存，但是不便于访问参数，不便于实现参数实时修改或者模型故障注入。

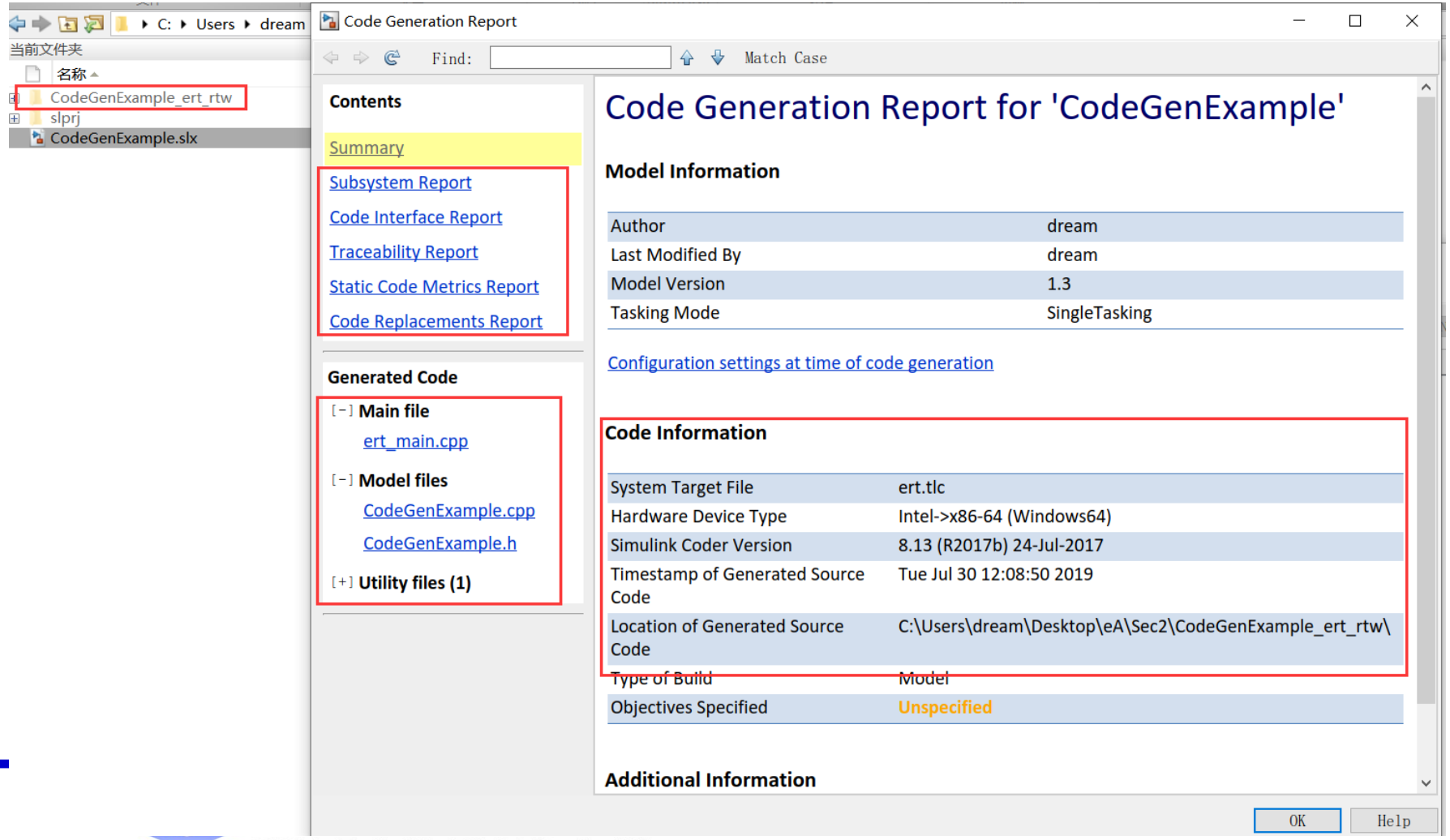


3.进阶例子



3.1 自主生成C/C++代码例子

- 点击编译按钮
- 生成三个文件
- 分别是：
- ert_main.cpp该文件包含一个调用生成代码的例子
- ****.cpp和****.h这两个文件包含了刚才的Simulink项目生成的一个C++类



3.进阶例子

3.1 自主生成C/C++代码例子

- 右图为生成的C++类（****.h文件中）：****ModelClass
- ****_P为参数结构体
- ****_U为输入结构体
- ****_Y为输出结构体
- step()为单步更新函数
- initializie()为初始化函数
- terminate()为终止函数

```
144 // Class declaration for model CodeGenExample
145 class CodeGenExampleModelClass {
146   // public data and function members
147  public:
148   // Tunable parameters
149   P_CodeGenExample_T CodeGenExample_P;
150
151   // External inputs
152   ExtU_CodeGenExample_T CodeGenExample_U;
153
154   // External outputs
155   ExtY_CodeGenExample_T CodeGenExample_Y;
156
157   // model initialize function
158   void initialize();
159
160   // model step function
161   void step();
162
163   // model terminate function
164   void terminate();
165
166   // Constructor
167   CodeGenExampleModelClass();
168
169   // Destructor
170   ~CodeGenExampleModelClass();
171
```

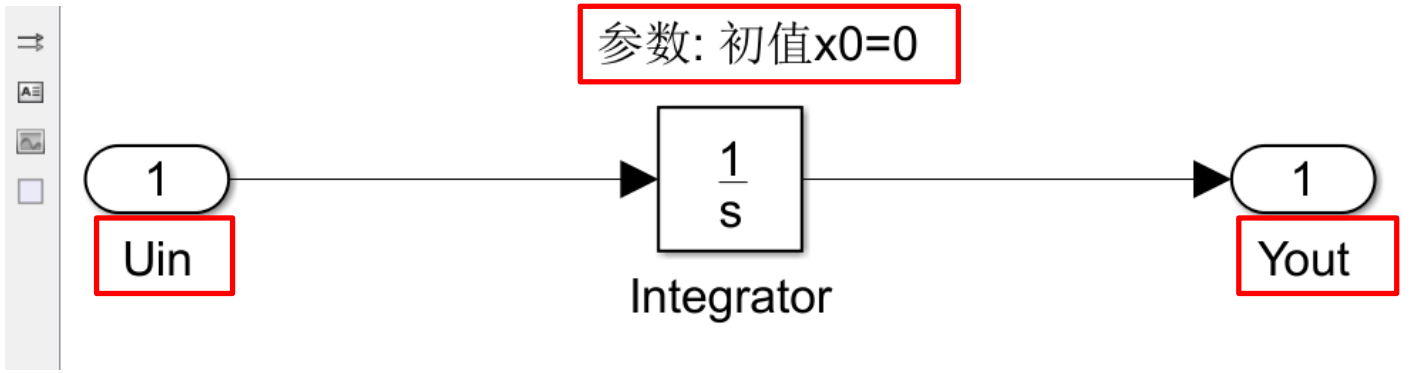
```
90 // Parameters (auto storage)
91 struct P_CodeGenExample_T_ {
92   real_T X0;                          // Variable: X0
93                                       //  Referenced by: '<Root>/Integrator'
94
95 };
96
97 // Parameters (auto storage)
98 typedef struct P_CodeGenExample_T_ P_CodeGenExample_T;
99
```

3.进阶例子

```
80 typedef struct {
81   real_T Uin;                          // '<Root>/Uin'
82 } ExtU_CodeGenExample_T;
83
84 // External outputs (root outports fed by signals with auto st
85 typedef struct {
86   real_T Uout;                         // '<Root>/Uout'
87 } ExtY_CodeGenExample_T;
88
```

3.1 自主生成C/C++代码例子

- ert_main.cpp文件的运行框架
- 该文件需要用户自行编写
- 在程序运行开始前，新建一个TestModelClass实例，并初始化
- 例如：TestModelClass m_testClass; m_testClass. initializie();
- 生成一个中断或者定时器，每0.001s调用一次回调函数，在该函数中进行如下操作:1.更新输入信息；2.更新参数信息；3.调用step()函数；4.更新输出信息。
- m_testClass. Test_U. Uin=***;
- m_testClass. Test_P.X0=***;
- m_testClass.step();
- ***= m_testClass. Test_Y.Yout;
- 退出时m_testClass. Terminate();



3.进阶例子

3.2 Pixhawk代码生成工具箱生成代码解析

本平台PX4代码相对原始固件进行的改动总结：

1、在“Firmware\cmake\configs****.cmake”文件中，添加“modules/px4_simulink_app”语句。

2、在“Firmware\src\modules\”目录下建立“px4_simulink_app”文件夹，和emptyfile.c+CMakeLists.txt。

3、在“Firmware\ROMFS\px4fmu_common\init.d\ rcS”文件中添加开机启动指令：px4_simulink_app start

nuttx_px4fmu-v4_default.cmake

```
include(nuttx/px4_impl_nuttx)

set(CMAKE_TOOLCHAIN_FILE ${CMAKE_

set(config_uavcan_num_ifaces 1)

set(config_module_list
  #
  # Board support modules
  #
  modules/px4_simulink_app
  drivers/device
```

```
# Boot is complete, inform MAVLi
mavlink boot_complete
px4_simulink_app start
if [ $EXIT_ON_END == yes ]
then
  echo "NSH EXIT"
  exit
fi
unset EXIT_ON_END
```

empty_file.c

```
// this is a place-holder file to complete the build process

__EXPORT int px4_simulink_app_main(int argc, char *argv[]);

int px4_simulink_app_main(int argc, char *argv[]) //Px4 App
{

  return 0;
}
```

CMakeLists.txt

```
## This is a place-holder cmakelist.txt file
## It will get replaced by the Pixhawk PSP

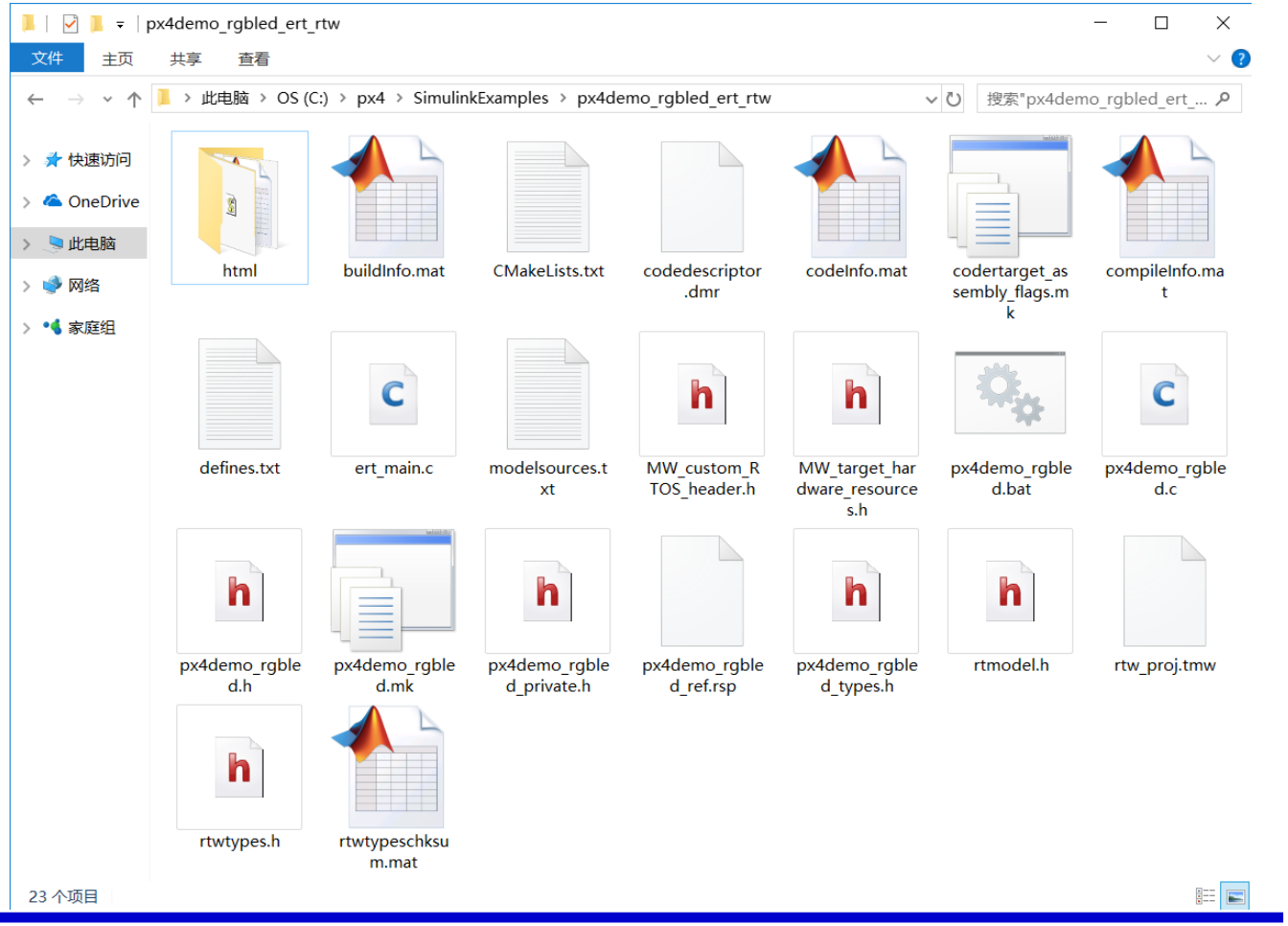
px4_add_module(
MODULE modules__px4_simulink_app
MAIN px4_simulink_app
STACK_MAIN 2000
SRCS
   empty_file.c
DEPENDS
    platforms__common
)
```

2020/8/12

3.进阶例子

3.2 Pixhawk代码生成工具箱生成代码解析

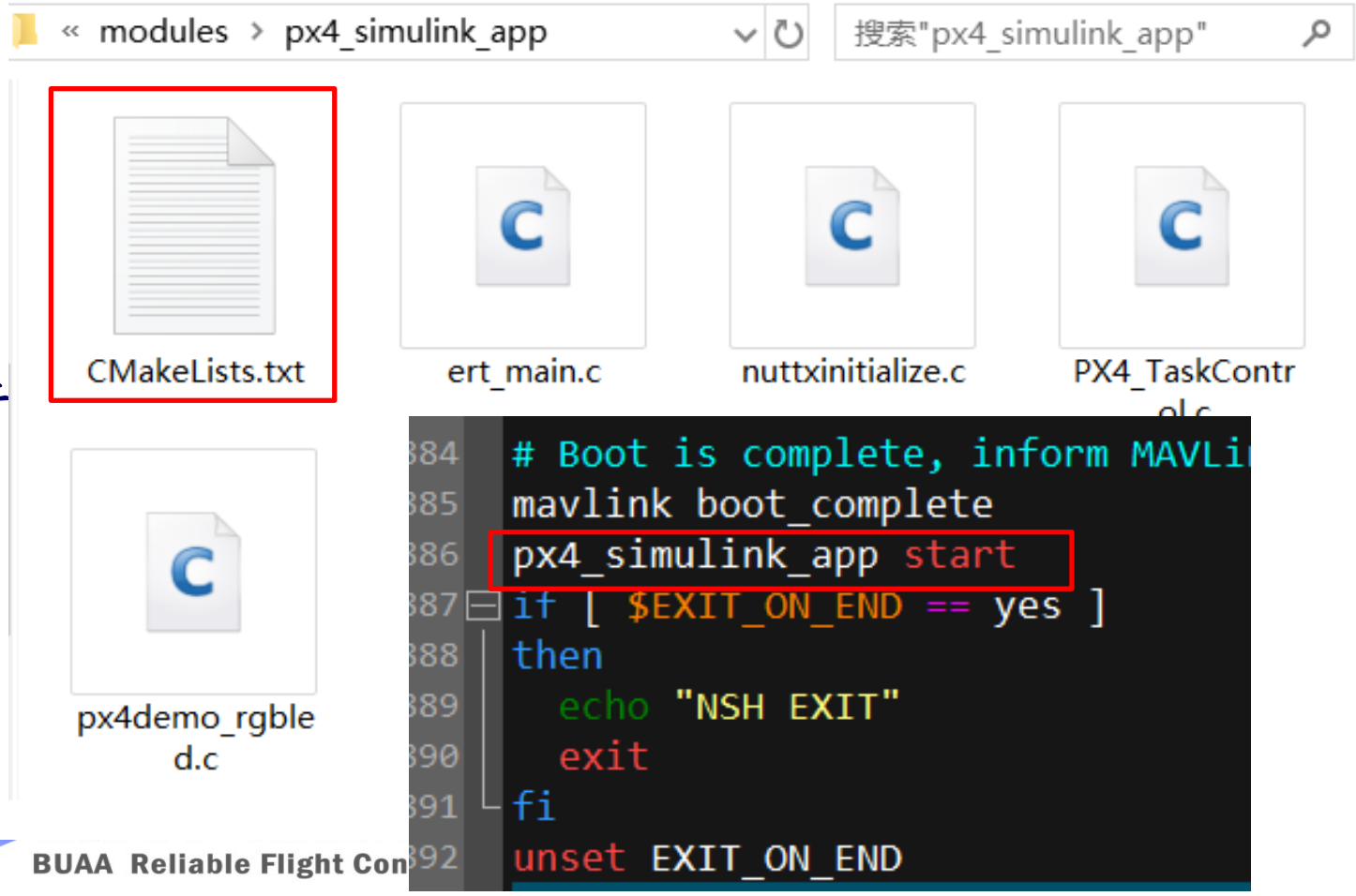
- 任意打开一个slx例程文件（例如LED例程）编译后生成的ert_rtw后缀文件夹，生成的主要文件包括:
- CMakeLists.txt
- ert_main.c
- ***.h
- ***.c
- ***.mk 该文件用于在Matlab完成代码生成后，将代码拷贝到合适位置(px4_simulink_ app文件夹)，并调用PX4编译指令编译固件



3.进阶例子

3.2 Pixhawk代码生成工具箱生成代码解析

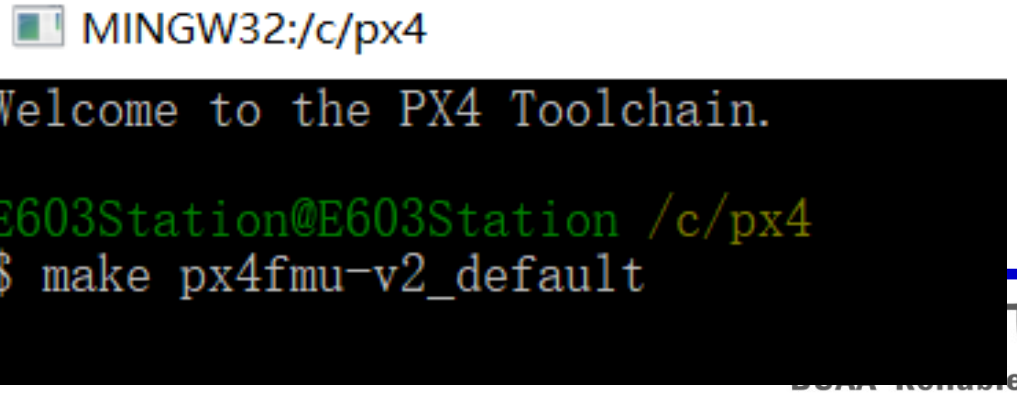
- 进入“Firmware\src\modules\px4_simulink_app\”文件夹
- 可以看到生成的关键文件
- 其中，CMakeLists.txt用于PX4编译器将刚刚生成的控制代码编译为一个px4_simulink_app的线程
- 注：默认情况下这个线程不会开自启，需要在Firmware\ROMFS\px4fmu_common\init.d\rcS文件添加开机启动指令：px4_simulink_app start
- 注：平台安装脚本已自动添加



3.进阶例子

3.2 Pixhawk代码生成工具箱生成代码解析

- PX4固件编译原理(以PX4 1.7固件fmu-v2为例):
- 1、打开编译器Win10WSL/Cygwin/Msys2
- 2、输入make px4fmu-v2_default
- 该指令会调用cmake去打开cmake\configs\nuttx_px4fmu-v2_default.cmake文件（PX4 1.9及之后版本会调用Firmware\boards\px4\fmu-v2\default）
- 3、编译px4_simulink_app文件夹



nuttx_px4fmu-v2_default.cmake

```
include(nuttx/px4_impl_nuttx)

set(CMAKE_TOOLCHAIN_FILE ${CMAK

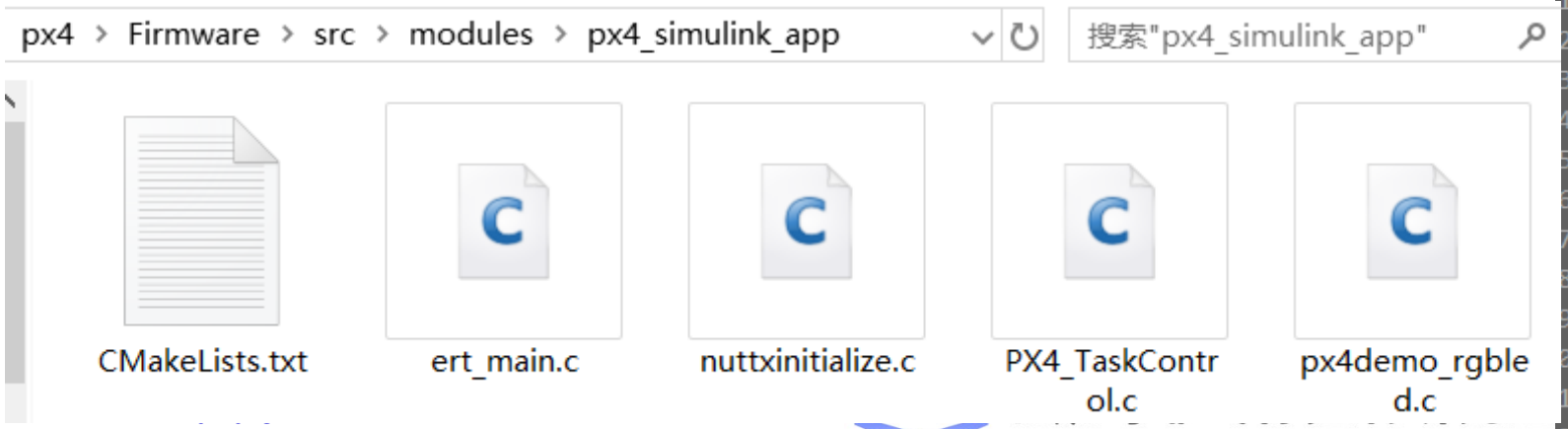
set(config_uavcan_num_ifaces 2)

set(config_module_list
  #
  # Board support modules
  #
  modules/px4_simulink_app
  drivers/device
  drivers/stm32
  drivers/stm32/adc
  drivers/stm32/tone_alarm
  drivers/led
  drivers/px4fmu
  drivers/px4io
  drivers/boards/px4fmu-v2
  drivers/rgbled
  drivers/mpu6000
  drivers/mpu9250
  drivers/lsm303d
  drivers/l3gd20
  drivers/hmc5883
  drivers/ms5611
```

3.进阶例子

3.2 Pixhawk代码生成工具箱生成代码解析

- PX4固件编译原理:
- 4、在px4_simulink_app文件夹找到CmakeLists.txt文件
- 该文件中定义了编译该app线程的方式
- 首先是包含源文件的路径
- 其次是该app的主要依赖文件和线程优先级



CMakeLists.txt

```
## This cmakelist.txt file was generated from On
## the Pixhawk Pilot Support Package

add_definitions(
-DMODEL=px4demo_rgbled -DNUMST=1 -DNCSTATES=0 -D

include_directories(
"D:/MATLAB/R2016b/simulink/include/sf_runtime"
"C:/px4/SimulinkExamples/px4demo_rgbled_ert_rtw"
"C:/px4/SimulinkExamples"
"D:/MATLAB/R2016b/extern/include"
"D:/MATLAB/R2016b/simulink/include"
"D:/MATLAB/R2016b/rtw/c/src"
"D:/MATLAB/R2016b/rtw/c/src/ext_mode/common"
"D:/MATLAB/R2016b/rtw/c/ert"
"D:/MATLAB/R2016b/rtw/c/src/ext_mode/serial"
"C:/Users/E603Station/Documents/MATLAB/Add-Ons/T
)

px4_add_module(
MODULE modules__px4_simulink_app
MAIN px4_simulink_app
STACK_MAIN 2000
SRCS
    ert_main.c
    px4demo_rgbled.c
    PX4_TaskControl.c
    nuttxinitialize.c
DEPENDS
    platforms__common
)
```

3.进阶例子

3.3 Pixhawk代码生成工具箱模块编程

https://www.mathworks.com/help/pdf_doc/rtw/index.html

Simulink Coder Getting Started Guide

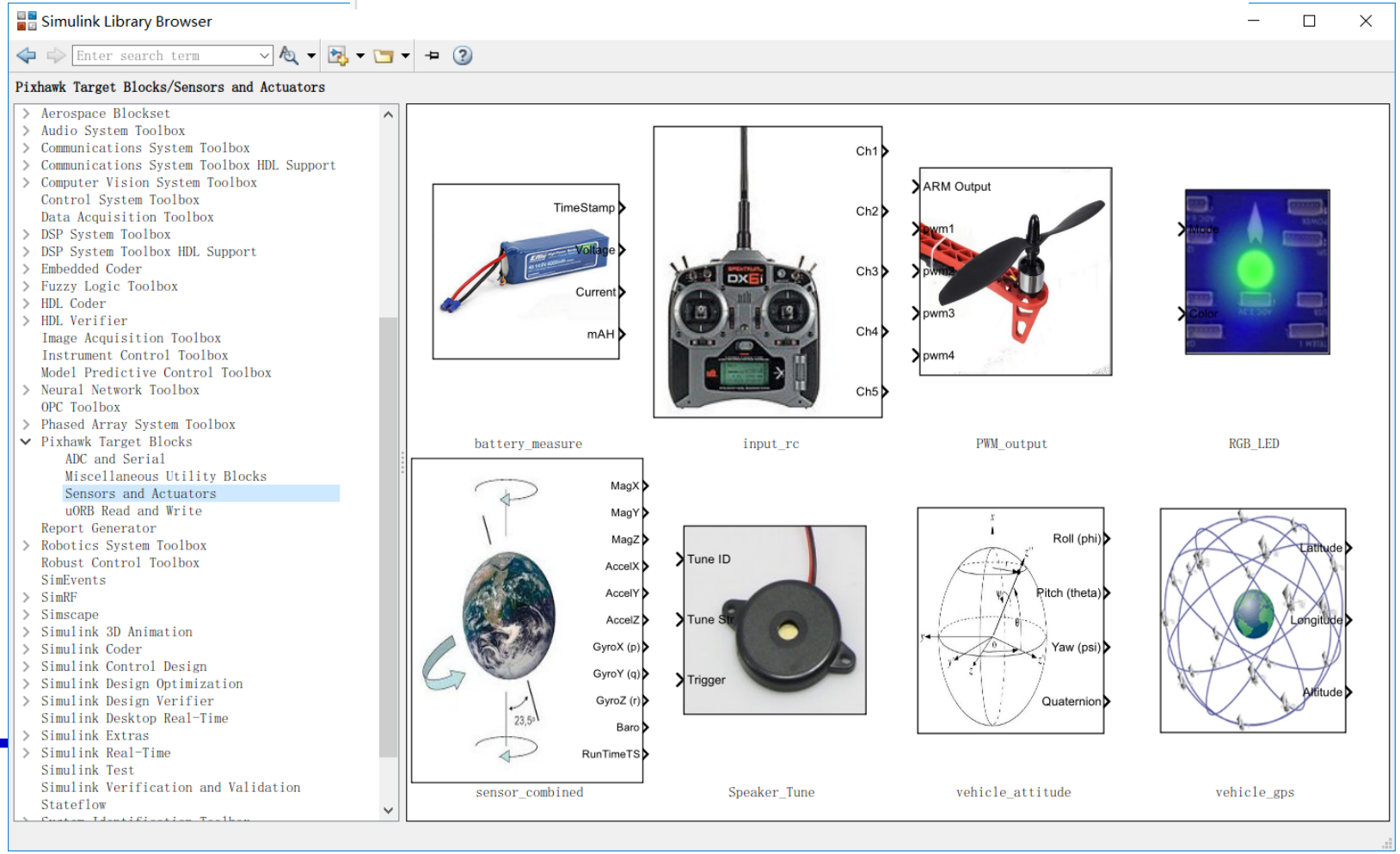
Simulink Coder User's Guide

Simulink Coder Target Language Compiler

Simulink Coder Reference

Simulink Coder Release Notes

- 这些模块都是由S函数加上tlc文件组成
- 其中，tlc文件为代码生成模板，定义了该模块如何生成代码来访问PX 4的驱动接口，来与底层硬件交互信息
- tlc文件的格式可以参考Matlab相关教程



3.进阶例子

3.3 Pixhawk代码生成工具箱模块编程

- 从Simulink模块属性中获取s函数（tlc）位置

【文档】\MATLAB\Add-Ons\Toolboxes\PX4PSP\code\blocks

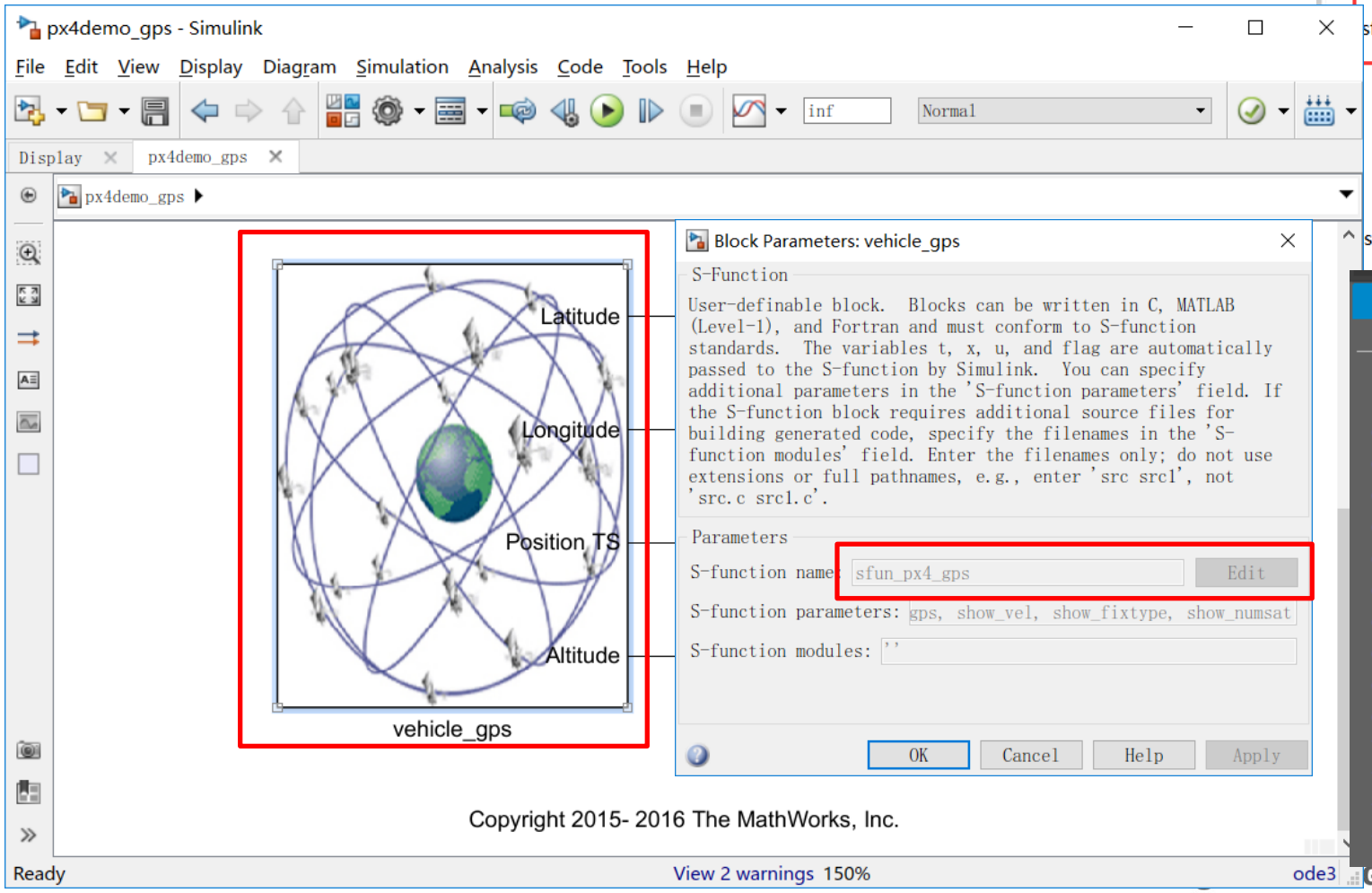
主页 共享 查看

此电脑 › 文档 › MATLAB › Add-Ons › Toolboxes › PX4 PSP › code › blocks

sfun_px4_aux.mexa64 sfun_px4_aux.mexw64 sfun_px4_aux.tlc sfun_px4_battery.mexa64 sfun_px4_battery.mexw64

sfun_px4_gps.mexa64 sfun_px4_gps.mexw64 sfun_px4_gps.tlc sfun_px4_input_rc.mexa64 sfun_px4_input_rc.mexw64

sfun_px4_pwm.mexa64 sfun_px4_pwm.mexw64 sfun_px4_pwm.tlc sfun_px4_rgbled.mexa64 sfun_px4_rgbled.mexw64



sfun_px4_gps.tlc

```
58 %% it is in the global rtB structure.
59 %%
60 %function Start(block, system) Output
61   %if  LibBlockOutputSignalIsInBlockIO(0)
62     %assign dW = LibBlockDWork(DWork[0],"","",0)
63     {
64     /* %<Type> Block: %<Name> */
65     /* subscribe to gps topic */
66   int fd = orb_subscribe(ORB_ID(vehicle_gps_position));
67     %<dW>.fd = fd;
68     %<dW>.events = POLLIN;
69   orb_set_interval(fd, 1);
70     warnx("* Subscribed to gps topic (fd = %d)*\n", fd);
71     }
72   %endif
73 %endfunction %% Start
```

C++订阅gps的uORB消息

Group

谢谢！